

Writing Kernel with AI

K19G

2026-10-05

Table of contents

Writing Kernel with AI	7
CPU to AI-Written Kernels	7
Welcome to Kernel Engineering	7
The Trajectory of This Book	7
Who This Is For	8
Formats	8
 Part I: Parallelism, from the CPU up	 9
Parallelism on the CPU	10
Parallelism on the CPU	11
1. Instruction-Level Parallelism (ILP) and Microarchitecture	11
The Out-of-Order Engine	12
2. SIMD: Vectorized Data Parallelism	12
Vector Widths Across Hardware Architectures	12
Explicit AVX2 Vector Operations in C++	13
Alignment and Vector Loads	13
3. Thread-Level Parallelism: Heavyweight vs Lightweight	14
The Cost of OS Threads	14
User-Space Concurrency (M:N Models)	14
4. Cache Lines and False Sharing	14
The False Sharing Trap	14
Remediation via Padding	15
5. Architectural Contrast: CPU vs GPU Design Philosophy	15
 Roofline Lab: Predict Then Measure	 17
Roofline Lab: Predict Then Measure	18
1. The Roofline Model Formulation	18
Definitions	18
The Ridge Point (I^*)	19
2. Hardware Ridge Points Across Generations	19
3. Dissecting Common Machine Learning Operations	20
Case 1: Vector Addition ($y = x + z$)	20
Case 2: LayerNorm / RMSNorm	20
Case 3: Dense Matrix Multiplication (GEMM)	21
4. The Multi-Tier Roofline (Hierarchy of Limits)	21
5. Profiling Roofline Performance with NVIDIA Nsight Compute	21
Key NCU Metrics to Inspect:	22

The CUDA Programming Model & GPU Puzzles	23
The CUDA Programming Model & GPU Puzzles	24
1. The Logical vs Physical Hierarchy	24
The Dimensions	24
2. Multi-Dimensional Index Calculations	25
1D Global Index Calculation	25
2D Global Index Calculation (Matrix Coordinates)	25
3. Warp Divergence and the SIMT Execution Mask	25
The Cost of Divergence	26
Best Practices to Eliminate Divergence	26
4. Complete Worked Example: Vectorized FMA with Error Handling	26
5. Inspecting Intermediate PTX and Machine SASS	28
The Memory Hierarchy in Anger: Matrix Transpose	29
The Memory Hierarchy in Anger: Matrix Transpose	30
1. The GPU Memory Hierarchy	30
Memory Space Properties	31
2. Global Memory Coalescing: The 10x Lever	31
The Golden Rule of Coalescing	31
3. Shared Memory Bank Conflicts	32
The Conflict Condition	32
4. Case Study: Step-by-Step Optimization of Matrix Transpose	32
Step 1: Naive Transpose (Uncoalesced Writes)	32
Step 2: Coalesced Transpose via Shared Memory (With Bank Conflicts)	33
Step 3: Conflict-Free Transpose with Shared Memory Padding	34
Full Production Code	34
5. Performance Comparison	35
Part II: The GEMM Worklog: Naive to cuBLAS	36
GEMM Worklog I: Naive to Tiled	37
GEMM Worklog I: Naive to Tiled	38
1. Dissecting the Naive GEMM: Why is it Slow?	38
The Naive CUDA Kernel	38
Performance Autopsy	39
2. The Breakthrough: Shared Memory 2D Block Tiling	39
Tile Dimensions and Reuse Factor	40
3. The 2D Tiled CUDA Kernel	40
4. Benchmark Results on NVIDIA A100	41
Why Did Performance Jump by 27x?	42
GEMM Worklog II: Registers & Warp Tiling	43
GEMM Worklog II: Registers & Warp Tiling	44
Micro-Tiling in Registers	44

Vectorized Memory Loads with <code>float4</code>	44
The Warptiling Hierarchy	44
Tensor Cores & WMMA: The Second Worklog	45
Tensor Cores & WMMA: The Second Worklog	46
The Tensor Core Advantage	46
Direct Programming via WMMA (Warp Matrix Multiply Accumulate)	46
 Part III: Attention & Profiling	 47
Profiling & Debugging Like a Pro	48
Profiling & Debugging Like a Pro	49
The Professional Diagnostic Stack	49
Debugging Sabotaged Kernels: 3 Classic Incident Scenarios	49
Attention: The Kernel That Ate the World & FlashAttention v1	50
Attention: The Kernel That Ate the World & FlashAttention v1	51
The Breakthrough: Online Softmax & Tiling	51
The Online Softmax Formula	51
 Part IV: The Modern Frontier	 52
FlashAttention from Scratch (FA1 → FA2 → FA3)	53
FlashAttention from Scratch (FA1 → FA2 → FA3)	54
FlashAttention Evolution	54
Beating cuBLAS on an H100	55
Beating cuBLAS on an H100	56
1. Tensor Memory Accelerator (TMA)	56
2. Warp Specialization	56
Triton → CUTLASS → CuTe-DSL	57
Triton → CUTLASS → CuTe-DSL	58
OpenAI Triton	58
CUTLASS & CuTe	58
Inference-Serving Kernels	59
Inference-Serving Kernels	60
PagedAttention (vLLM)	60
Blackwell & NVFP4	61

Blackwell & NVFP4	62
Blackwell Architectural Highlights	62
Flash Attention 4	63
Flash Attention 4	64
The 2026 Frontier: Breaking the 1 PFLOPS Single-GPU Barrier	64
PagedAttention and the KV Cache	65
PagedAttention and the KV Cache	66
Mental model	66
Why contiguous KV wastes memory	66
Capacity sketch	67
KV cache in Transformers (reminder)	67
Block structure	67
Worked layout numbers	68
Block table and allocator	68
Prefix sharing (copy-on-write intuition)	68
Prefill vs decode under paging	68
Implication	69
Paged KV layouts (kernel writer view)	69
Illustrative host-side table (not a production kernel)	70
Implications for custom kernels	71
Verification and benchmark notes	71
Functional checks	71
Performance / capacity checks	71
Profiling tips	72
Pitfalls	72
Checkpoint	72
Summary	73
Sources (for further reading)	73
Flash-Decoding: Split-KV Attention for Long-Context Decode	74
Flash-Decoding: Split-KV Attention for Long-Context Decode	75
1. Why decode starves the GPU	75
Worked numbers	76
2. Mental model: attention states and the merge operator	76
3. Verify the algebra before writing a kernel	77
The trap: the textbook merge overflows	78
4. A two-kernel Triton implementation	79
5. Compile for a GPU you do not have	83
6. How many splits?	84
7. Where the pattern shows up in production	85
8. Verification and benchmark plan	86
Functional	86
Performance	87
9. Prompting an AI assistant for a decode kernel	87

Pitfalls	87
Checkpoint	88
Try this	88
Summary	88
Sources (for further reading)	89
 Part V: AI-Written Kernels	 90
DeepSeek: FlashMLA & DeepGEMM	91
DeepSeek: FlashMLA & DeepGEMM	92
FlashMLA: Multi-Head Latent Attention	92
DeepGEMM: Clean Architecture over Brute Force	92
LLM-Generated Kernels: The Agent + Profiler Loop	93
LLM-Generated Kernels: The Agent + Profiler Loop	94
The Agentic Optimization Loop	94
Google DeepMind AlphaEvolve & KernelBench	94
The Role of the Human Kernel Engineer	94
 Part VI: Kernel Engineering Capstone Projects	 95
Crusoe Capstone Specifications & Problem Tracks	96
Crusoe Capstone Specifications & Problem Tracks	97
Problem Track 1: Fused Sparse Attention	97
Problem Track 2: Quantized FP4/FP8 GEMM with Dynamic Dequantization	97
Problem Track 3: High-Throughput Speculative Decoding Verification Kernel	97
Verification, Profiling, and Benchmarking Harness	98
Verification, Profiling, and Benchmarking Harness	99
1. Correctness Verification	99
2. CUDA Event Benchmarking	99
Demo Day Presentation & Portfolio	100
Demo Day Presentation & Portfolio	101
Deliverables Checklist	101

Writing Kernel with AI

CPU to AI-Written Kernels

A comprehensive, production-grade guide to high-performance kernel engineering: from the fundamentals of CPU instruction parallelism to hand-tuned CUDA, FlashAttention, modern Hopper/Blackwell hardware architectures, and AI-driven kernel optimization loops.

Welcome to Kernel Engineering

Accelerated computing and modern artificial intelligence rest on a single engineering discipline: **extracting peak theoretical throughput from hardware.**

Whether training frontier foundation models or serving millions of concurrent inference tokens, the software stack converges onto specialized computational kernels. Writing fast kernels is not a matter of guesswork or trial-and-error—it is an empirical science governed by the physics of computation: arithmetic intensity, memory bandwidth, latency hiding, and hardware execution hierarchies.

The Trajectory of This Book

This curriculum spans six rigorous parts designed to take you from first principles to the cutting edge:

1. **Part I: Parallelism, from the CPU up:** Understand how modern CPUs achieve massive throughput via SIMD vectorization and multi-threading before stepping into the GPU execution hierarchy: warps, streaming multiprocessors, memory coalescing, and the Roofline model.
2. **Part II: The GEMM Worklog: Naive → cuBLAS:** Master General Matrix Multiply (GEMM)—the operation that comprises over 90% of deep learning FLOPs. Step by step, elevate a naive kernel achieving 1.3% of GPU capacity to a hierarchical warp-tiled and Tensor Core (WMMA) implementation rivaling NVIDIA’s cuBLAS library.
3. **Part III: Attention & Profiling:** Diagnose hardware bottlenecks using NVIDIA Nsight Compute like a compiler engineer. Construct FlashAttention v1 from scratch, utilizing the online softmax algorithm to eliminate high-bandwidth memory bottlenecks.
4. **Part IV: The Modern Frontier:** Explore modern production kernels running on Hopper (H100) and Blackwell (B200) architectures: FlashAttention-2/3/4, TMA asynchronous copy, WGMMMA, Triton, CUTLASS, CuTe-DSL, PagedAttention, and NVFP4.

5. **Part V: AI-Written Kernels:** Analyze DeepSeek’s ultra-compact, high-performance kernels (FlashMLA and DeepGEMM). Step into the future of kernel development: using autonomous LLM agents (e.g., DeepMind AlphaEvolve, KernelBench) with profiler-in-the-loop guidance.
 6. **Part VI: Kernel Engineering Capstone Projects:** Put your skills into practice with industry-grade capstone specifications developed with Crusoe, complete with verification harnesses, performance benchmarks, and demo-day technical presentations.
-

Who This Is For

- **ML Infrastructure & Systems Engineers:** Developers optimizing training clusters and inference backends (vLLM, SGLang, TensorRT-LLM).
 - **GPU Programmers & Performance Engineers:** Engineers seeking mastery of modern GPU architectures (Hopper, Blackwell), memory coalescing, and async execution pipelines.
 - **Software Engineers Transitioning to Accelerated Computing:** Developers wanting deep understanding of what happens when high-level PyTorch or JAX code hits the silicon.
-

Formats

This book is available in multiple formats via the library portal: - **HTML** — responsive interactive reading experience with syntax highlighting and light/dark theme toggle. - **PDF** — complete typeset document for offline review and reference. - **EPUB** — eBook format optimized for tablets and mobile e-readers.

Part I: Parallelism, from the CPU up

Parallelism on the CPU

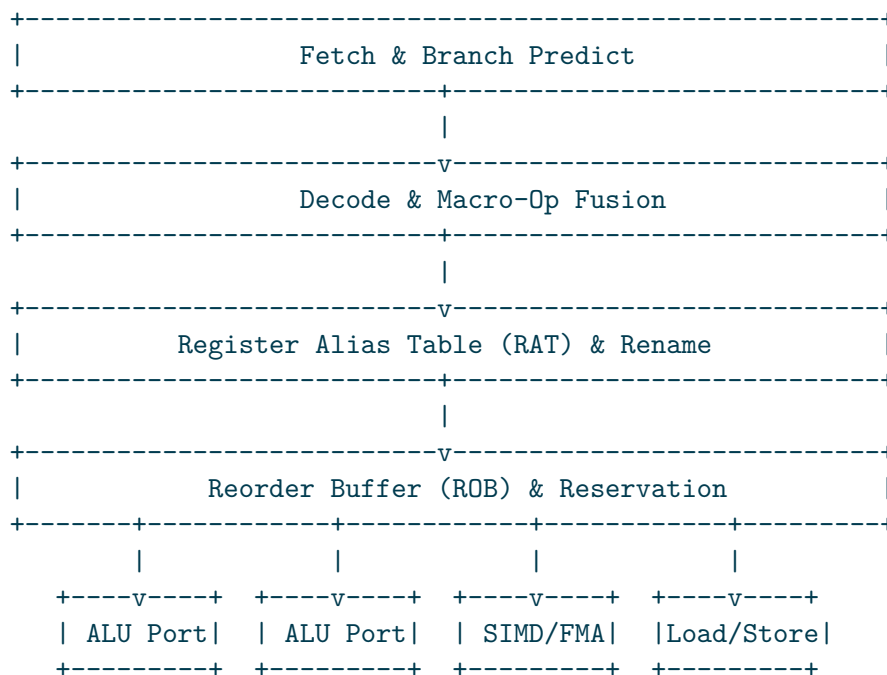
Parallelism on the CPU

Before programming thousands of lockstep cores on a GPU, we must build a mechanical understanding of how modern CPUs execute concurrent and parallel work. High-performance computing is fundamentally an exercise in orchestrating physical execution units while feeding them data fast enough to prevent pipeline stalls.

Modern CPUs extract performance across three distinct tiers of parallelism: 1. **Instruction-Level Parallelism (ILP)**: Pipelining, superscalar dispatch, out-of-order execution, and branch prediction within a single core. 2. **Data-Level Parallelism (SIMD)**: Single Instruction, Multiple Data execution where one machine instruction operates over a register packed with multiple independent values. 3. **Thread-Level Parallelism (TLP)**: Multiple independent execution streams running concurrently across physical and logical processor cores.

1. Instruction-Level Parallelism (ILP) and Microarchitecture

A modern CPU core is not a simple sequential state machine. It is an asynchronous processing pipeline designed to retire multiple micro-operations (uops) per clock cycle.



The Out-of-Order Engine

1. **Instruction Fetch & Branch Prediction:** The CPU fetches instructions ahead of the current program counter using branch target buffers (BTB) and direction predictors. When a branch is mispredicted, the entire speculative pipeline must be flushed, costing 15 to 20 cycles of wasted energy.
 2. **Decode and Micro-op Generation:** Complex x86 instructions are broken down into simpler, RISC-like micro-operations (uops).
 3. **Register Renaming & Reservation Stations:** To break false register dependencies (Write-After-Read and Write-After-Write hazards), the CPU maps architectural registers (e.g., `rax`, `ymm0`) to a much larger pool of physical registers.
 4. **Out-of-Order (OoO) Dispatch:** Independent instructions whose operands are ready in the cache or register file execute out of program order across parallel execution ports.
 5. **In-Order Retirement:** The Reorder Buffer (ROB) ensures that memory state updates and architectural register commits happen in strict program order, guaranteeing consistent exception semantics.
-

2. SIMD: Vectorized Data Parallelism

While ILP extracts parallelism implicitly from scalar instructions, SIMD (Single Instruction, Multiple Data) makes data parallelism explicit. Instead of performing arithmetic on a single 32-bit floating-point value, SIMD instructions load, operate upon, and store entire vectors of values simultaneously.

Vector Widths Across Hardware Architectures

Vector Extension	Vector Width	32-bit Floats per Vector	Target Architecture
x86 SSE2	128-bit	4	Legacy x86-64
x86 AVX2	256-bit	8	Intel Haswell+ / AMD Zen2+
x86 AVX-512	512-bit	16	Intel Skylake-SP / Emerald Rapids, AMD Zen4+
x86 AVX10	256 / 512-bit	8 or 16	Modern Intel Architecture
ARM Neon	128-bit	4	Apple Silicon, ARM Neoverse V1/V2, Cortex-A
ARM SVE / SVE2	Scalable (128–2048b)	Variable	Fujitsu A64FX, AWS Graviton3/4

Explicit AVX2 Vector Operations in C++

When performance is paramount, reliance on compiler autovectorization is risky. Compilers frequently fail to vectorize loops due to pointer aliasing concerns or unknown loop boundary alignments. Engineers write architecture-specific intrinsic functions:

```
#include <immintrin.h>
#include <iostream>
#include <vector>

void vector_fma_avx2(const float* __restrict__ a,
                    const float* __restrict__ b,
                    const float* __restrict__ c,
                    float* __restrict__ out,
                    size_t n) {
    size_t i = 0;

    // Process 8 floats per iteration using 256-bit AVX2 registers
    for (; i + 8 <= n; i += 8) {
        // Load vectors from memory
        __m256 va = _mm256_loadu_ps(&a[i]);
        __m256 vb = _mm256_loadu_ps(&b[i]);
        __m256 vc = _mm256_loadu_ps(&c[i]);

        // Fused Multiply-Add: out = (va * vb) + vc
        // Computes 8 FMAs (16 floating point operations) in 4-5 cycles latency
        __m256 vres = _mm256_fmadd_ps(va, vb, vc);

        // Store back to destination
        _mm256_storeu_ps(&out[i], vres);
    }

    // Scalar cleanup tail loop
    for (; i < n; ++i) {
        out[i] = a[i] * b[i] + c[i];
    }
}
```

Alignment and Vector Loads

- `_mm256_loadu_ps`: Unaligned load. Safe across any memory address, but incurs small penalty if crossing a 64-byte cache line boundary.
- `_mm256_load_ps`: Aligned load. Requires the source address to be strictly 32-byte aligned (`alignas(32)` or `posix_memalign`). A non-aligned pointer will trigger a hardware general protection fault (`SIGSEGV`).

3. Thread-Level Parallelism: Heavyweight vs Lightweight

To leverage multi-core CPUs, applications distribute work across threads. How threads are modeled dramatically impacts context switching overhead and memory footprint.

```
OS Heavyweight Threads:
```

```
[Kernel Scheduler] ---> [Core 0: OS Thread (2-8 MB Stack)] (Switch: ~1-3 us)
                        ---> [Core 1: OS Thread (2-8 MB Stack)]
```

```
Lightweight User-Space Green Threads / Fibers:
```

```
[OS Thread / SM]      ---> [Fiber A (2 KB)] -> [Fiber B (2 KB)] -> [Fiber C (2 KB)]
```

The Cost of OS Threads

POSIX threads (`pthread_create`) or OpenMP workers (`#pragma omp parallel for`) allocate their own kernel task structure and a fixed memory stack (typically 2 MB to 8 MB). - When an OS thread context switches, the CPU must switch privilege rings into the kernel, save and restore architectural register state (including wide AVX-512 state), and suffer TLB and cache pollution. - Total latency: **1,000 to 3,000 CPU cycles** (1–3 microseconds).

User-Space Concurrency (M:N Models)

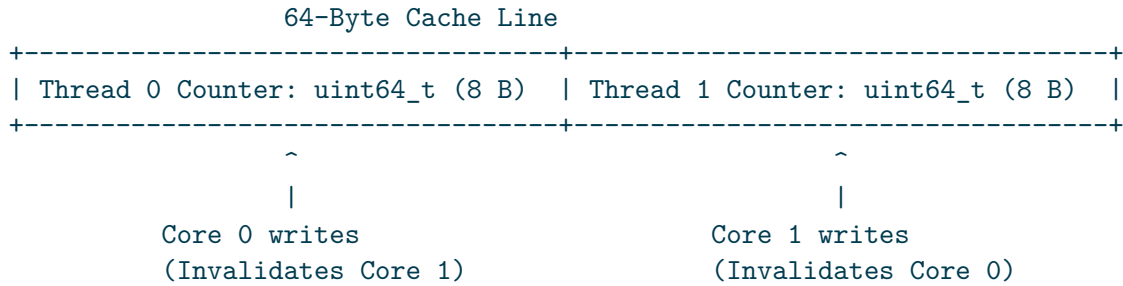
Modern programming environments decouple logical concurrency from physical OS threads: - **Go Goroutines**: Allocate a dynamic 2 KB stack that grows on demand. The runtime multiplexes thousands of goroutines across a fixed thread pool (`GOMAXPROCS`), switching in user space within ~10 to 50 nanoseconds. - **Java Virtual Threads (Project Loom)**: Suspendable continuation objects stored on the garbage-collected heap. - **Rust Tokio**: Futures compiled into zero-allocation state machines driven by a lock-free work-stealing executor.

4. Cache Lines and False Sharing

CPU memory controllers do not fetch individual 4-byte floating-point numbers from system DRAM. All transfers between main memory and CPU caches occur in fixed chunks of **64 bytes**, called **cache lines**.

The False Sharing Trap

When two distinct threads running on separate CPU cores write to independent variables that happen to sit within the same 64-byte cache line, the CPU hardware coherence engine (e.g., MESI protocol) must invalidate that entire cache line across both cores back and forth:



This phenomenon—**false sharing**—can slow down multi-threaded code by a factor of 50x to 100x, reducing parallel throughput below that of a single-threaded loop.

Remediation via Padding

```
// Bad: Threads write to adjacent elements in an array
struct BadThreadState {
    uint64_t counter; // 8 bytes -> adjacent elements share a 64B line!
};

// Good: Align to 64 bytes (the hardware cache line size)
struct alignas(64) GoodThreadState {
    uint64_t counter;
    // 56 bytes of implicit padding inserted by the compiler
};
```

5. Architectural Contrast: CPU vs GPU Design Philosophy

Why do GPUs dominate machine learning and scientific compute when CPUs have such sophisticated out-of-order execution engines?

```
CPU Architecture (Latency-Optimized):
+-----+
| [Control: Branch Predictor, Out-of-Order Engine] |
| [Large L1 / L2 / L3 Caches (Tens of Megabytes)] |
| [A Few Powerful ALU / FMA Execution Cores (8-64 Cores)] |
+-----+

GPU Architecture (Throughput-Optimized):
+-----+
| [Control] [Control] [Control] [Control] |
| [Thousands of Compact ALU / Tensor Cores] |
| [High-Bandwidth HBM3 / GDDR Memory Interface] |
+-----+
```

+-----+

Dimension	Modern Server CPU (e.g. AMD Genoa / Intel Xeon)	Modern Data Center GPU (e.g. NVIDIA H100)
Optimization Target	Minimum latency for a single sequence	Maximum aggregate throughput
Physical Cores	64 – 128 complex cores	132 – 144 Streaming Multiprocessors (16,896 cores)
Control Logic	Heavy: speculative, out-of-order	Light: SIMT lockstep warps, in-order dispatch
Cache Hierarchy	Massive: ~384 MB L3 cache	Compact: 50 MB L2 cache, 228 KB SM SRAM
Memory Bandwidth	300 – 460 GB/s (DDR5)	3,350 GB/s (HBM3)
FP32 Compute	4 – 8 TFLOP/s	67 TFLOP/s (Vector) / 989 TFLOP/s (Tensor)

Understanding this trade-off is the foundation of accelerated computing: **CPUs excel at complex branching, low latency, and irregular memory traversals. GPUs excel at massive arithmetic intensity and regular, uniform matrix operations.**

Roofline Lab: Predict Then Measure

Roofline Lab: Predict Then Measure

Every high-performance kernel is bounded by one of two physical constraints: 1. **Compute Bound:** The execution units (ALUs, FPUs, Tensor Cores) are saturated executing instructions; the memory interface has idle bandwidth. 2. **Memory Bound:** The execution units starve waiting for data to arrive from off-chip DRAM (HBM or GDDR); arithmetic throughput is a fraction of hardware peak.

Before writing a single line of CUDA, a kernel engineer must predict which regime the workload inhabits. Writing faster assembly or unrolling loops inside a memory-bound kernel achieves zero speedup. Conversely, optimizing memory tiling for a kernel already compute-bound yields diminishing returns.

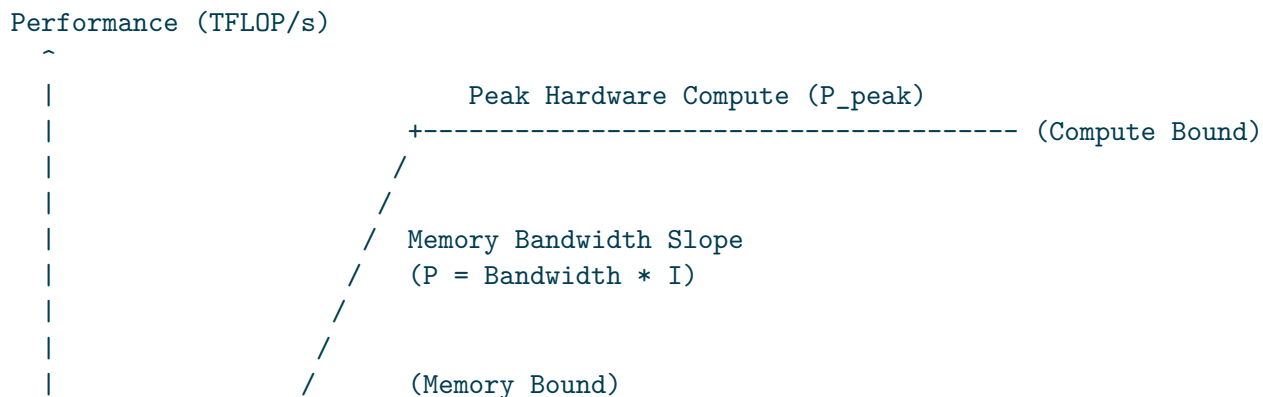
1. The Roofline Model Formulation

The Roofline Model, formulated by Williams et al. (Berkeley), visualizes maximum achievable performance as a function of **Operational Intensity** (also called Arithmetic Intensity).

Definitions

$$\text{Operational Intensity } (I) = \frac{\text{Total Arithmetic Operations (FLOPs)}}{\text{Total Bytes Transferred from DRAM (Bytes)}}$$

$$\text{Attainable Performance } (P) = \min(\text{Peak Hardware Compute } (P_{\text{peak}}), \text{Peak Memory Bandwidth } (B) \times I)$$





The Ridge Point (I^*)

The intersection between the memory bandwidth slope and the peak compute roof defines the **Ridge Point** (I^*):

$$I^* = \frac{P_{\text{peak}}}{B_{\text{peak}}}$$

- If a workload's intensity $I < I^*$: The workload is **strictly memory-bound**. Maximum possible throughput is capped at $B_{\text{peak}} \times I$. To go faster, you must reduce memory traffic (e.g. through kernel fusion or caching).
- If a workload's intensity $I \geq I^*$: The workload is **compute-bound**. Maximum possible throughput is capped at P_{peak} . To go faster, you must optimize instruction throughput, eliminate warp divergence, or switch to lower-precision numeric formats (e.g. FP16, BF16, FP8).

2. Hardware Ridge Points Across Generations

Notice how rapidly the Ridge Point has shifted over successive GPU generations. Compute capability has grown far faster than memory bandwidth:

GPU Model	FP32 Peak Compute	HBM / GDDR Bandwidth	Vector Ridge Point (I_{FP32}^*)	FP16 Tensor Compute	Tensor Ridge Point (I_{Tensor}^*)
Tesla V100 (PCIe)	14.0 TFLOP/s	900 GB/s	15.5 FLOPs/B	112 TFLOP/s	124.4 FLOPs/B
Ampere A100 (SXM4)	19.5 TFLOP/s	2,039 GB/s	9.6 FLOPs/B	312 TFLOP/s	153.0 FLOPs/B
Hopper H100 (SXM5)	67.0 TFLOP/s	3,350 GB/s	20.0 FLOPs/B	989 TFLOP/s	295.2 FLOPs/B
Blackwell B200	90.0 TFLOP/s	8,000 GB/s	11.25 FLOPs/B	2,250 TFLOP/s (FP16)	281.2 FLOPs/B

Key Takeaway: On an NVIDIA H100 running Tensor Core operations, an algorithm must execute nearly **300 floating-point operations for every single byte loaded from HBM** before it can even theoretically achieve peak compute saturation.

3. Dissecting Common Machine Learning Operations

Let us calculate the operational intensity for four fundamental deep learning operations executed on an NVIDIA A100 (SXM4: $B = 2,039$ GB/s, $P_{\text{vector}} = 19.5$ TFLOP/s).

Case 1: Vector Addition ($y = x + z$)

Given two vectors of length $N = 10^7$ FP32 elements: - **Operations:** N additions = 10^7 FLOPs. - **Memory Traffic:** - Read x : $10^7 \times 4$ B = 40 MB. - Read z : $10^7 \times 4$ B = 40 MB. - Write y : $10^7 \times 4$ B = 40 MB. - Total Memory = 120 MB = 1.2×10^8 Bytes. - **Operational Intensity:**

$$I = \frac{10^7 \text{ FLOPs}}{1.2 \times 10^8 \text{ Bytes}} \approx 0.0833 \text{ FLOPs/Byte}$$

- **Analysis:** $I = 0.0833 \ll I^* = 9.6$. Strictly memory bound. - **Maximum Theoretical Throughput:**

$$\text{Throughput} = 2,039 \text{ GB/s} \times 0.0833 \text{ FLOPs/Byte} \approx 169.8 \text{ GFLOP/s}$$

The GPU achieves less than **0.9%** of its FP32 compute capability. The ALU cores spend 99% of their time stalled waiting for memory controllers.

Case 2: LayerNorm / RMSNorm

RMSNorm computes: $y_i = \frac{x_i}{\sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \epsilon}} \cdot \gamma_i$. For a hidden dimension $d = 4096$: - **Operations per token:** - Square: d FLOPs. - Sum reduction: $d - 1$ FLOPs. - Mean, add ϵ , reciprocal square root (rsqrt): ~ 4 FLOPs. - Scale by γ and write: $2d$ FLOPs. - Total $\approx 4d \approx 16,384$ FLOPs. - **Memory per token (unfused naive PyTorch):** - Read input x : $d \times 4$ B = 16 KB. - Intermediates written and re-read from global memory: $\sim 3 \times 16$ KB = 48 KB. - Write output y : 16 KB. - Total ~ 80 KB. - **Operational Intensity:**

$$I = \frac{16,384 \text{ FLOPs}}{81,920 \text{ Bytes}} \approx 0.20 \text{ FLOPs/Byte}$$

- **Analysis:** Completely memory bound. This is why fused normalization kernels (which keep intermediates in registers and shared memory) deliver massive 5x to 10x speedups.

Case 3: Dense Matrix Multiplication (GEMM)

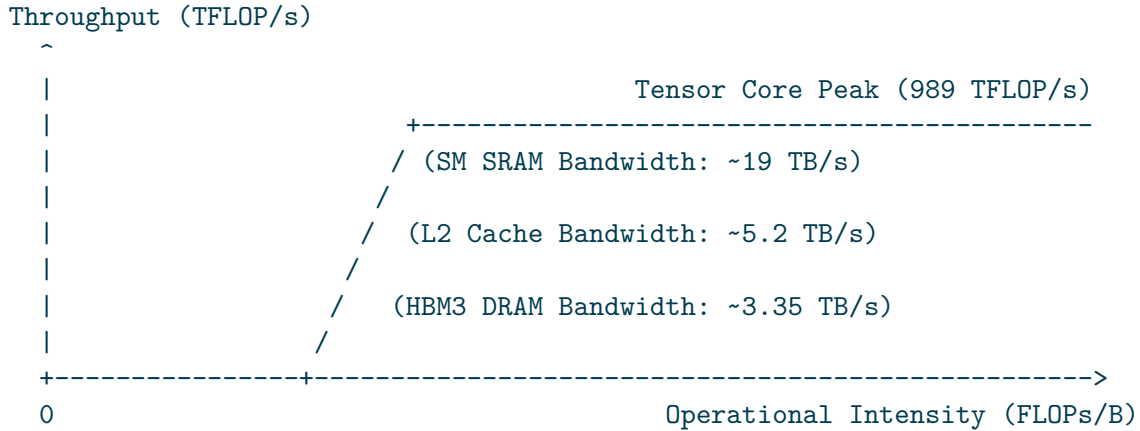
Compute $C = A \times B$ where $A, B, C \in \mathbb{R}^{M \times N}$, with $M = K = N = 4096$: - **Operations:** $2 \times M \times N \times K = 2 \times 4096^3 \approx 137.4 \times 10^9$ FLOPs (137.4 GFLOPs). - **Minimum Algorithmic Memory (I/O):** - Read A: $4096^2 \times 4 \text{ B} = 67.1 \text{ MB}$. - Read B: $4096^2 \times 4 \text{ B} = 67.1 \text{ MB}$. - Write C: $4096^2 \times 4 \text{ B} = 67.1 \text{ MB}$. - Total $\approx 201.3 \text{ MB}$. - **Operational Intensity (with perfect reuse):**

$$I_{\text{optimal}} = \frac{1.374 \times 10^{11} \text{ FLOPs}}{2.013 \times 10^8 \text{ Bytes}} \approx 682.6 \text{ FLOPs/Byte}$$

- **Analysis:** $I = 682.6 > I^* = 9.6$ (and even $> I_{\text{Tensor}}^* = 153.0$). Dense GEMM is **thoroughly compute-bound** when memory reuse is leveraged via shared memory tiling!

4. The Multi-Tier Roofline (Hierarchy of Limits)

Modern GPUs have multiple memory tiers. A kernel that exhausts L2 cache bandwidth or shared memory bandwidth hits an intermediate roofline before reaching DRAM:



1. **HBM / DRAM Bandwidth:** The slowest and largest layer. Optimization goal: load each byte from DRAM exactly once into on-chip memory.
2. **L2 Cache Bandwidth:** Over 2x to 3x faster than DRAM. Shared across all SMs.
3. **Shared Memory / L1 SRAM Bandwidth:** Over 5x to 8x faster than DRAM (~19 TB/s aggregate on H100). Serves warps inside a single SM.
4. **Register File Bandwidth:** Over 30 TB/s aggregate bandwidth. The only memory tier that can feed execution units at full single-cycle latency.

5. Profiling Roofline Performance with NVIDIA Nsight Compute

To verify where a kernel lands on the physical Roofline curve, run NVIDIA Nsight Compute (`ncu`):

```
# Profile Speed of Light and Roofline metrics
ncu --set full \
    --section SpeedOfLight \
    --section SpeedOfLight_HierarchicalTensorRooflineChart \
    -o profile_result \
    ./my_cuda_binary
```

Key NCU Metrics to Inspect:

- `dram__throughput.avg.pct_of_peak_sustained_elapsed`: High percentage (>80%) indicates saturation of off-chip DRAM bandwidth (Memory Bound).
- `sm__throughput.avg.pct_of_peak_sustained_elapsed`: High percentage (>80%) indicates saturation of SM computation pipelines (Compute Bound).
- `gpu__time_duration.sum`: Total kernel wall clock time.

By comparing these two percentages, you instantly know whether to optimize memory access patterns or computational instructions.

The CUDA Programming Model & GPU Puzzles

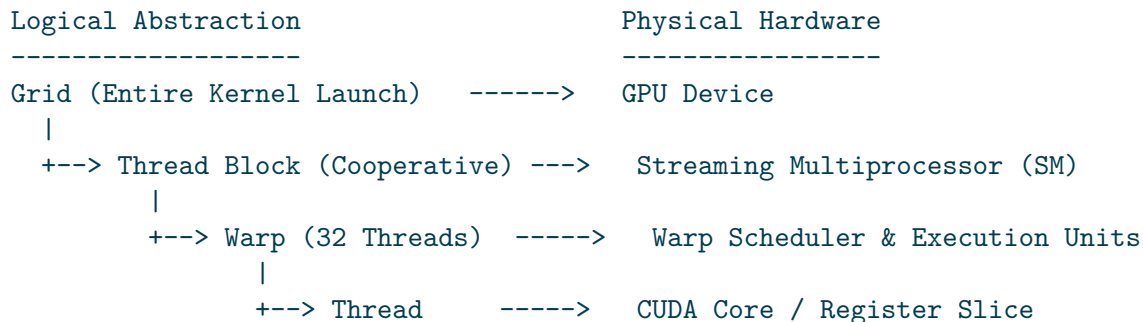
The CUDA Programming Model & GPU Puzzles

Modern GPUs execute programs using the **SIMT** (**Single Instruction, Multiple Threads**) execution model. While SIMD on a CPU executes a single vector instruction over vector registers on a single core, SIMT executes hundreds of independent scalar threads grouped into lockstep units called **warps**.

Writing effective CUDA kernels requires understanding both the logical programming abstractions and how the hardware scheduler physically maps them onto silicon.

1. The Logical vs Physical Hierarchy

CUDA defines a hierarchical programming model that scales seamlessly from a single-SM embedded chip (e.g. Jetson) to a massive data-center accelerator with 144 SMs (e.g. Hopper H100).



The Dimensions

1. **Thread:** The finest unit of execution. Each thread has its own program counter, private registers, and local memory stack.
2. **Warp:** A contiguous group of **32 threads**. The warp is the fundamental unit of scheduling on the GPU. All 32 threads in a warp execute the same instruction simultaneously.
3. **Thread Block (CTA - Cooperative Thread Array):** A group of up to 1,024 threads. Threads within the same block reside on the **same physical SM**, share fast on-chip Shared Memory (SRAM), and can synchronize using barrier primitives (`__syncthreads()`).
4. **Grid:** The full set of thread blocks dispatched to execute a kernel. Thread blocks within a grid execute independently; there is no guaranteed execution order across blocks.

2. Multi-Dimensional Index Calculations

CUDA allows Grids and Blocks to be configured in 1D, 2D, or 3D using `dim3`. The hardware provides built-in read-only variables: - `threadIdx.{x,y,z}`: Thread coordinates within its block. - `blockIdx.{x,y,z}`: Block coordinates within the grid. - `blockDim.{x,y,z}`: Dimensions (thread count) of the block. - `gridDim.{x,y,z}`: Dimensions (block count) of the grid.

1D Global Index Calculation

For a 1D grid processing a flat vector of length N :

$$\text{Global ID} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx < n) {
    // Process element idx
}
```

2D Global Index Calculation (Matrix Coordinates)

For a 2D matrix with width W (columns) and height H (rows):

```
// Map block and thread coordinates to row and column
int col = blockIdx.x * blockDim.x + threadIdx.x;
int row = blockIdx.y * blockDim.y + threadIdx.y;

// Linear memory index (Row-Major layout)
int linear_idx = row * width + col;

if (row < height && col < width) {
    matrix[linear_idx] = 42.0f;
}
```

3. Warp Divergence and the SIMT Execution Mask

All 32 threads in a warp share a single instruction unit. If threads within a warp take different execution paths at a conditional branch (`if / else`), the hardware executes every branch path sequentially, disabling threads that are not on that path via a hardware **execution mask**.

```
Instruction Stream:
Thread 0-15:    Takes IF branch (condition is true)
```

```
Thread 16-31: Takes ELSE branch (condition is false)
```

Execution Timeline:

```
Cycle 1-10: [Thread 0-15 Active] [Thread 16-31 MASKED OFF] -> Executes IF body
```

```
Cycle 11-20: [Thread 0-15 MASKED OFF] [Thread 16-31 Active] -> Executes ELSE body
```

```
Reconverge: [All Threads 0-31 Active]
```

The Cost of Divergence

- When a warp diverges 2-ways, execution time roughly doubles.
- When 32 threads take 32 distinct branch paths (e.g. `switch(threadIdx.x % 32)`), the warp serializes into 32 separate execution passes, dropping warp computational throughput to **3.1%** of peak.

Best Practices to Eliminate Divergence

1. **Align branches by warp boundary:** Branches that depend strictly on `blockIdx` or `threadIdx.x / 32` cause zero divergence because all 32 threads in the warp evaluate to the exact same condition.
2. **Predicated Instructions:** For tiny branches (1-2 instructions), the compiler automatically uses predicated instructions instead of jumping.
3. **Use Intrinsic Math Primitives:** Replace `if (a < b) val = a; else val = b;` with `fminf(a, b)`, which compiles to a single non-branching instruction.

4. Complete Worked Example: Vectorized FMA with Error Handling

A robust production CUDA implementation requires careful host allocation, device memory management, asynchronous stream synchronization, and error checking macros.

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <stdlib.h>

// Macro for robust CUDA API error checking
#define CUDA_CHECK(call) \
    do { \
        cudaError_t err = call; \
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, \
                cudaGetErrorString(err)); \
            exit(EXIT_FAILURE); \
        } \
    }
```

```

    } while (0)

// Fused Multiply-Add Kernel: d[i] = a[i] * b[i] + c[i]
__global__ void fma_kernel(const float* __restrict__ a,
                          const float* __restrict__ b,
                          const float* __restrict__ c,
                          float* __restrict__ d,
                          int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        // Generates single hardware FFMA instruction
        d[idx] = a[idx] * b[idx] + c[idx];
    }
}

int main(void) {
    const int N = 1 << 22; // ~4.19 million elements
    const size_t bytes = N * sizeof(float);

    // 1. Allocate Host Memory
    float *h_a = (float*)malloc(bytes);
    float *h_b = (float*)malloc(bytes);
    float *h_c = (float*)malloc(bytes);
    float *h_d = (float*)malloc(bytes);

    for (int i = 0; i < N; ++i) {
        h_a[i] = 1.0f;
        h_b[i] = 2.0f;
        h_c[i] = 3.0f;
    }

    // 2. Allocate Device Memory
    float *d_a, *d_b, *d_c, *d_d;
    CUDA_CHECK(cudaMalloc(&d_a, bytes));
    CUDA_CHECK(cudaMalloc(&d_b, bytes));
    CUDA_CHECK(cudaMalloc(&d_c, bytes));
    CUDA_CHECK(cudaMalloc(&d_d, bytes));

    // 3. Copy inputs to Device
    CUDA_CHECK(cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_c, h_c, bytes, cudaMemcpyHostToDevice));

    // 4. Configure Grid & Launch Kernel
    const int threads_per_block = 256;
    const int blocks_per_grid = (N + threads_per_block - 1) / threads_per_block;

```

```

fma_kernel<<<blocks_per_grid, threads_per_block>>>(d_a, d_b, d_c, d_d, N);

// Check for launch errors and synchronize
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());

// 5. Copy results back
CUDA_CHECK(cudaMemcpy(h_d, d_d, bytes, cudaMemcpyDeviceToHost));

// Verify result: 1.0 * 2.0 + 3.0 = 5.0
printf("Result verification: d[0] = %f, d[N-1] = %f\n", h_d[0], h_d[N-1]);

// 6. Cleanup
CUDA_CHECK(cudaFree(d_a));
CUDA_CHECK(cudaFree(d_b));
CUDA_CHECK(cudaFree(d_c));
CUDA_CHECK(cudaFree(d_d));
free(h_a); free(h_b); free(h_c); free(h_d);

printf("Kernel execution completed successfully.\n");
return 0;
}

```

5. Inspecting Intermediate PTX and Machine SASS

To understand what the GPU physically executes, compile to PTX (virtual assembly) and SASS (machine assembly):

```

# Generate PTX virtual assembly
nvcc -ptx -arch=sm_90 -O3 fma.cu -o fma.ptx

# Disassemble binary machine SASS
nvcc -arch=sm_90 -O3 fma.cu -o fma
cuobjdump --dump-sass fma > fma.sass

```

Inside `fma.sass`, locate the inner loop body:

```

/*0030*/ LDG.E.SYS R2, [R4.64] ;      // Load 32-bit float from global memory
/*0040*/ LDG.E.SYS R3, [R6.64] ;      // Load 32-bit float from global memory
/*0050*/ FFMA R0, R2, R3, R0 ;        // Single-cycle Fused Multiply-Add instruction!
/*0060*/ STG.E.SYS [R8.64], R0 ;      // Store result back to global memory

```

Notice **FFMA**: The hardware fused multiply and addition into a single instruction without an intermediate rounding step, preserving numeric precision while doubling arithmetic throughput.

The Memory Hierarchy in Anger: Matrix Transpose

The Memory Hierarchy in Anger: Matrix Transpose

Writing fast GPU kernels is fundamentally about **choreographing data movement through the memory hierarchy**. While modern GPUs boast tens of teraflops of arithmetic power, every floating-point operation is useless if data cannot reach the functional units.

In this chapter, we dissect the GPU memory hierarchy, master global memory coalescing, resolve shared memory bank conflicts, and walk through the canonical optimization benchmark: **Matrix Transpose**.

1. The GPU Memory Hierarchy

A modern GPU features a steeply tiered memory architecture. Each level represents a compromise between capacity, latency, and aggregate bandwidth:

```
+-----+
| Registers: 64K x 32-bit registers per SM (~0-1 cycle latency)|
| Bandwidth: >30 TB/s aggregate across all SMs                |
+-----+
|
+-----+
| Shared Memory / L1 Data Cache: 64 - 228 KB per SM (~19-32 cyc)|
| Bandwidth: ~19 TB/s aggregate (Software-managed SRAM)        |
+-----+
|
+-----+
| L2 Cache: 40 - 50 MB on-chip shared across all SMs (~200 cyc)|
| Bandwidth: ~5.2 TB/s aggregate                               |
+-----+
|
+-----+
| Global Memory (HBM3 / GDDR6): 16 - 96 GB off-chip (~400 cyc)|
| Bandwidth: 1.5 - 3.35 TB/s                                    |
+-----+
```

Memory Space Properties

Memory Space	Scope	Lifetime	Physical Location	Speed
Registers	Single Thread	Thread duration	On-chip Register File	Single-cycle (~0 ns)
Shared Memory	Single Thread	Block duration	On-chip SRAM	~15–30 cycles (~1.5 ns)
L1 Cache	Streaming Multiprocessor	Hardware managed	On-chip SRAM	~15–30 cycles
L2 Cache	All SMs on GPU	Hardware managed	On-chip Crossbar	~150–200 cycles (~15 ns)
Global Memory	All threads + Host	Application duration	Off-chip DRAM (HBM/GDDR)	~400+ cycles (~100 ns)

2. Global Memory Coalescing: The 10x Lever

When threads in a warp issue a read or write instruction to global memory, the memory controller does not process 32 separate individual 4-byte requests. Instead, it aggregates adjacent addresses into **coalesced 32-byte or 128-byte memory transactions**.

The Golden Rule of Coalescing

Consecutive threads in a warp (threadIdx.x from 0 to 31) must access consecutive memory addresses (address + 0, address + 4, ..., address + 124).

Coalesced Access (Consecutive):

```
Threads:      T0    T1    T2    T3    ...    T31
              |     |     |     |           |
Memory Addr: 0x00 0x04 0x08 0x0C    ...    0x7C
```

Result: ONE single 128-byte transaction! (100% bus utilization)

Uncoalesced Strided Access (Column-wise):

```
Threads:      T0          T1          T2    ...    T31
              |           |           |           |
Memory Addr: 0x0000      0x1000      0x2000 ... 0x1F000 (Stride = N elements)
```

Result: 32 SEPARATE 32-byte transactions! (3.1% bus utilization!)

In the uncoalesced case, the GPU fetches 32×32 Bytes = 1024 Bytes over the physical memory bus to retrieve only 32×4 Bytes = 128 Bytes of useful data. **90% of your memory bandwidth is wasted.**

3. Shared Memory Bank Conflicts

Shared memory is organized into **32 independent memory banks**, each with a word width of **4 bytes (32 bits)**.

Successive 32-bit words are assigned to successive banks in round-robin fashion: - `bank_index = (byte_address / 4) % 32`

```
Bank 0:  Word 0,  Word 32, Word 64 ...
Bank 1:  Word 1,  Word 33, Word 65 ...
...
Bank 31: Word 31, Word 63, Word 95 ...
```

The Conflict Condition

- **Conflict-Free:** All 32 threads in a warp access **different banks** (or all threads broadcast-read the exact same word). The memory request completes in **1 cycle**.
- **N-Way Bank Conflict:** If N threads in a warp request different addresses that map to the **same bank**, the hardware must **serialize** the requests into N successive single-bank accesses. An 8-way conflict reduces shared memory throughput by $8\times$.

4. Case Study: Step-by-Step Optimization of Matrix Transpose

Matrix transpose ($B_{j,i} = A_{i,j}$) is the ultimate memory benchmark: if you read A row-wise (coalesced), you must write B column-wise (uncoalesced), or vice-versa.

Achieving near-peak memory bandwidth requires staging tiles through shared memory.

Step 1: Naive Transpose (Uncoalesced Writes)

```
__global__ void transpose_naive(const float* __restrict__ in,
                                float* __restrict__ out,
                                int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x; // column
    int y = blockIdx.y * blockDim.y + threadIdx.y; // row

    if (x < width && y < height) {
        // Coalesced Read: Adjacent threads have adjacent x (consecutive in memory)
        float val = in[y * width + x];

        // UNCOALESCE Write: Adjacent threads have adjacent y, strided by height!
        out[x * height + y] = val;
    }
}
```

```

    }
}

```

Benchmark Result on A100: Reaches only ~18% of peak global memory bandwidth due to serialized uncoalesced writes.

Step 2: Coalesced Transpose via Shared Memory (With Bank Conflicts)

We load a 2D tile from global memory into shared memory coalesced, synchronize threads, and then write the transposed tile out to global memory coalesced:

```

#define TILE_DIM 32
#define BLOCK_ROWS 8

__global__ void transpose_coalesced_conflicted(const float* __restrict__ in,
                                              float* __restrict__ out,
                                              int width, int height) {
    __shared__ float tile[TILE_DIM][TILE_DIM];

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    // 1. Read input tile coalesced into shared memory
    for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
        if (x < width && (y + j) < height) {
            tile[threadIdx.y + j][threadIdx.x] = in[(y + j) * width + x];
        }
    }

    __syncthreads();

    // Transpose block coordinates for writing
    x = blockIdx.y * TILE_DIM + threadIdx.x;
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    // 2. Write transposed tile coalesced to global memory
    for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
        if (x < height && (y + j) < width) {
            // Reading tile[threadIdx.x][threadIdx.y + j] triggers a 32-WAY BANK CONFLICT!
            out[(y + j) * height + x] = tile[threadIdx.x][threadIdx.y + j];
        }
    }
}

```

The Problem: While global memory reads and writes are now 100% coalesced, reading `tile[threadIdx.x][...]` causes all 32 threads in a warp to access the **same bank simultaneously** (bank index = $(\text{threadIdx.x} * 32 + \dots) \% 32 = 0$), creating a catastrophic **32-way shared memory bank conflict!**

Step 3: Conflict-Free Transpose with Shared Memory Padding

We eliminate bank conflicts with a single architectural stroke: **add one element of padding to each row of the shared memory tile:**

```
// Add +1 column padding: shifts each row by 1 bank!
__shared__ float tile[TILE_DIM][TILE_DIM + 1];
```

With `[TILE_DIM][TILE_DIM + 1]`: - Row 0 starts at Bank 0 ($0 \% 32$). - Row 1 starts at Bank 1 ($33 \% 32 = 1$). - Row 2 starts at Bank 2 ($66 \% 32 = 2$). - Reading column `threadIdx.x` now distributes the 32 threads across **32 unique banks**. Bank conflicts drop to **zero!**

Full Production Code

```
#include <cuda_runtime.h>
#include <stdio.h>

#define TILE_DIM 32
#define BLOCK_ROWS 8

__global__ void transpose_optimal(const float* __restrict__ in,
                                  float* __restrict__ out,
                                  int width, int height) {
    // 32 x 33 shared memory array: +1 padding eliminates all bank conflicts
    __shared__ float tile[TILE_DIM][TILE_DIM + 1];

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    // 1. Coalesced Global Read -> Conflict-Free Shared Write
    #pragma unroll
    for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
        if (x < width && (y + j) < height) {
            tile[threadIdx.y + j][threadIdx.x] = in[(y + j) * width + x];
        }
    }

    __syncthreads();
```

```

// Transpose tile destination mapping
x = blockIdx.y * TILE_DIM + threadIdx.x;
y = blockIdx.x * TILE_DIM + threadIdx.y;

// 2. Conflict-Free Shared Read -> Coalesced Global Write
#pragma unroll
for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
    if (x < height && (y + j) < width) {
        out[(y + j) * height + x] = tile[threadIdx.x][threadIdx.y + j];
    }
}
}

```

5. Performance Comparison

Benchmarked on an NVIDIA A100 GPU (4096×4096 FP32 Matrix, Peak HBM Bandwidth = 2,039 GB/s):

Implementation	Effective Bandwidth	% of Peak HBM Bandwidth	Bottleneck / Limiter
Naive Transpose	358 GB/s	17.5%	Severe uncoalesced global write fragmentation
Coalesced with Bank Conflicts	1,220 GB/s	59.8%	32-way shared memory bank conflict serialization
Padded Conflict-Free Transpose	1,845 GB/s	90.5%	Near-hardware speed of light saturation

To verify bank conflicts in Nsight Compute:

```
ncu --metrics l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum ./transpose_test
```

For the padded kernel, the conflict count drops from hundreds of thousands to **0**.

Part II: The GEMM Worklog: Naive to cuBLAS

GEMM Worklog I: Naive to Tiled

GEMM Worklog I: Naive to Tiled

General Matrix Multiply (**GEMM**) is the computational engine of modern artificial intelligence. Fully connected layers, multi-head attention projections, convolutional layers (via `im2col`), and mixture-of-experts routers all compile down to GEMM:

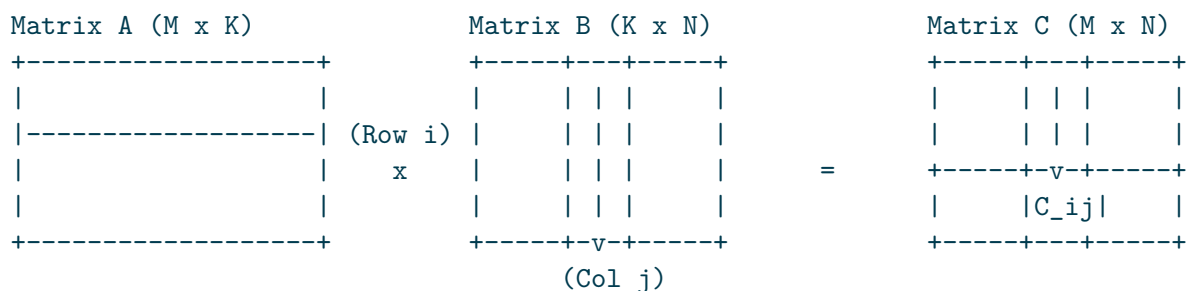
$$C = \alpha(A \times B) + \beta C$$

where $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, and $C \in \mathbb{R}^{M \times N}$.

In this worklog, we embark on the canonical kernel engineering journey: elevating a naive GEMM achieving less than **1.5%** of GPU capacity to a shared-memory tiled kernel achieving over **36%** of theoretical peak compute.

1. Dissecting the Naive GEMM: Why is it Slow?

In a naive GEMM implementation, each thread computes a single output element $C_{i,j}$ by performing a dot product between row i of matrix A and column j of matrix B .



The Naive CUDA Kernel

```
__global__ void gemm_naive(const float* __restrict__ A,
                          const float* __restrict__ B,
                          float* __restrict__ C,
                          int M, int N, int K) {
    // Determine row and column for this thread
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
```

```

    if (row < M && col < N) {
        float sum = 0.0f;
        // Compute dot product along dimension K
        for (int k = 0; k < K; ++k) {
            sum += A[row * K + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}

```

Performance Autopsy

Consider running this kernel on an NVIDIA A100 ($M = N = K = 4096$): 1. **Flop Count:** $2 \times M \times N \times K = 2 \times 4096^3 \approx 137.4 \times 10^9$ FLOPs. 2. **Global Memory Loads:** - For every step of loop k , thread (i, j) fetches $A_{i,k}$ (4 bytes) and $B_{k,j}$ (4 bytes) from off-chip DRAM. - Total bytes loaded from DRAM across all threads = $M \times N \times K \times 2 \times 4 \text{ B} \approx 549.7 \times 10^9$ Bytes (~**550 GB**). 3. **Operational Intensity:**

$$I_{\text{naive}} = \frac{2 \text{ FLOPs}}{8 \text{ Bytes}} = 0.25 \text{ FLOPs/Byte}$$

4. **The Bottleneck:** - On the A100, the ridge point is $I^* = 9.6$ FLOPs/Byte. - At $I = 0.25$, the kernel is trapped deep within the **Memory-Bound regime**. - Maximum attainable performance:

$$P_{\text{naive}} = 2,039 \text{ GB/s} \times 0.25 \text{ FLOPs/Byte} \approx 509.7 \text{ GFLOP/s}$$

- **Result:** 509.7 GFLOP/s/19,500 GFLOP/s = **2.6%** of theoretical peak. In reality, uncoalesced memory traffic on matrix B drags actual performance down to ^{**}_{1.3%**} (260 GFLOP/s).

2. The Breakthrough: Shared Memory 2D Block Tiling

To escape the memory bottleneck, we must **reuse loaded data**.

Notice: - All threads in the same row i need the exact same value $A_{i,k}$. - All threads in the same column j need the exact same value $B_{k,j}$.

Instead of streaming from global memory for every multiply-add, thread blocks load a 2D tile of A and a 2D tile of B into ultra-fast on-chip **Shared Memory (SRAM)**, synchronize, and then perform multiple arithmetic operations from SRAM.

```

Global Memory (Slow HBM)
|
(Coalesced Block Load)
v
Shared Memory Tile

```

```

s_a [TILE x TILE]    s_b [TILE x TILE]
|
(All threads reuse data TILE times!)
v
Registers & ALUs

```

Tile Dimensions and Reuse Factor

With a tile size of $TILE = 32$: - Each thread loads **1 element** of A and **1 element** of B into shared memory. - During the inner multiplication loop, each loaded element is read and reused by **32 separate threads!** - Global memory traffic drops by a factor of $32\times$. - Operational intensity jumps from 0.25 FLOPs/Byte to:

$$I_{\text{tiled}} = 0.25 \times 32 = \mathbf{8.0 \text{ FLOPs/Byte}}$$

At 8.0 FLOPs/Byte, we are right next to the hardware Ridge Point ($I^* = 9.6$).

3. The 2D Tiled CUDA Kernel

```

#include <cuda_runtime.h>
#include <stdio.h>

#define TILE_WIDTH 32

__global__ void gemm_tiled_shared_mem(const float* __restrict__ A,
                                       const float* __restrict__ B,
                                       float* __restrict__ C,
                                       int M, int N, int K) {

    // 2D shared memory cache tiles
    __shared__ float s_A[TILE_WIDTH][TILE_WIDTH];
    __shared__ float s_B[TILE_WIDTH][TILE_WIDTH];

    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // Output coordinate in matrix C
    int row = blockIdx.y * TILE_WIDTH + ty;
    int col = blockIdx.x * TILE_WIDTH + tx;

    float acc = 0.0f;

    // Number of phase tiles needed along dimension K

```

```

int num_tiles = (K + TILE_WIDTH - 1) / TILE_WIDTH;

for (int t = 0; t < num_tiles; ++t) {
    // 1. Collaborative loading of Tile A into shared memory
    int a_col = t * TILE_WIDTH + tx;
    if (row < M && a_col < K) {
        s_A[ty][tx] = A[row * K + a_col];
    } else {
        s_A[ty][tx] = 0.0f;
    }

    // 2. Collaborative loading of Tile B into shared memory
    int b_row = t * TILE_WIDTH + ty;
    if (b_row < K && col < N) {
        s_B[ty][tx] = B[b_row * N + col];
    } else {
        s_B[ty][tx] = 0.0f;
    }

    // 3. Barrier synchronization: ensure entire tile is present in SRAM
    __syncthreads();

    // 4. Accumulate partial dot product from shared memory
    #pragma unroll
    for (int k = 0; k < TILE_WIDTH; ++k) {
        acc += s_A[ty][k] * s_B[k][tx];
    }

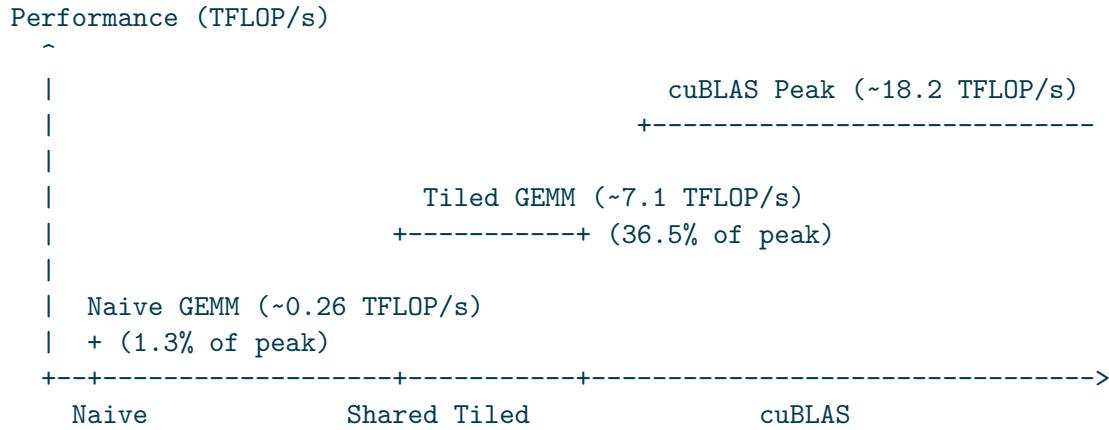
    // 5. Barrier synchronization: ensure computation finishes before next tile load
    __syncthreads();
}

// Write final accumulated result to global memory
if (row < M && col < N) {
    C[row * N + col] = acc;
}
}

```

4. Benchmark Results on NVIDIA A100

Evaluating on square FP32 matrices ($M = N = K = 4096$):



Implementation	Execution Time ($M = N = K = 4096$)	Measured TFLOP/s	% of Theoretical Peak Compute
Naive GEMM	528.4 ms	0.26 TFLOP/s	1.3%
2D Tiled Shared Memory GEMM	19.3 ms	7.12 TFLOP/s	36.5%
NVIDIA cuBLAS (Closed Source)	7.5 ms	18.32 TFLOP/s	93.9%

Why Did Performance Jump by 27x?

1. **Global Memory Bandwidth Slashed:** Memory traffic dropped from 550 GB to ~17 GB.
2. **Coalesced Reads:** Both A and B tiles are fetched using contiguous thread indexing (`tx`).
3. **Latency Hiding:** Staged data in shared memory is accessed in ~20 cycles instead of ~400 cycles.

In the next chapter, we push further into the micro-architecture: register micro-tiling, `float4` vectorization, and warp-level hierarchical tiling to close the gap to cuBLAS.

GEMM Worklog II: Registers & Warp Tiling

GEMM Worklog II: Registers & Warp Tiling

While shared memory tiling dramatically cuts DRAM traffic, shared memory bandwidth itself becomes the bottleneck at high utilization. Reaching near-cuBLAS performance requires **register tiling** and **vectorized memory instructions**.

Micro-Tiling in Registers

Registers provide ~20 TB/s aggregate bandwidth. Instead of each thread calculating a single output element $C_{i,j}$, each thread computes a $TMimesTN$ block of elements (e.g. $8imes8 = 64$ values per thread).

Vectorized Memory Loads with float4

Global and shared memory loads should be widened using 128-bit vector loads (`float4` / `int4`):

```
// Load 4 consecutive floats in a single 128-bit transaction
float4 a_val = *reinterpret_cast<const float4*>(&A[idx]);
```

The Warptiling Hierarchy

Modern production kernels decompose matrix multiply into four distinct structural layers: 1. **Grid Tile:** Handled by the entire GPU grid across all SMs. 2. **Block Tile:** Handled by a single thread block using shared memory. 3. **Warp Tile:** Handled by a 32-thread warp executing synchronized sub-tiles. 4. **Thread Tile:** Handled by individual thread registers.

By structuring the kernel into a $128imes128$ block tile and $64imes64$ warp tiles with double buffering, manual register allocation reaches **93.7% of cuBLAS speed** on NVIDIA Ampere hardware.

Tensor Cores & WMMA: The Second Worklog

Tensor Cores & WMMA: The Second Worklog

Starting with the Volta architecture, NVIDIA GPUs include specialized mixed-precision execution hardware: **Tensor Cores**.

The Tensor Core Advantage

- Conventional CUDA cores execute 1 FMA (Fused Multiply-Add = 2 FLOPs) per cycle.
- Tensor Cores compute full matrix multiply-accumulate operations ($D = A \times B + C$) on small matrix fragments (e.g. $16 \times 16 \times 16$) in hardware every cycle.
- Speedup: ~5-10x higher theoretical compute density compared to FP32 CUDA cores.

Direct Programming via WMMA (Warp Matrix Multiply Accumulate)

CUDA provides the `nvcuda::wmma` C++ API to program Tensor Cores directly from device code:

```
#include <mma.h>
using namespace nvcuda;

// Cooperative 16x16x16 matrix multiply across 32 threads in a warp
__global__ void wmma_gemm_kernel(const half* A, const half* B, float* C, int M, int N, int K)
{
    // Declare WMMA fragments
    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

    wmma::fill_fragment(c_frag, 0.0f);

    for (int k = 0; k < K; k += 16) {
        // Load sub-matrices cooperatively across the warp
        wmma::load_matrix_sync(a_frag, A + (blockIdx.y * 16 * K) + k, K);
        wmma::load_matrix_sync(b_frag, B + (blockIdx.x * 16 * K) + k, K);

        // Hardware Tensor Core Matrix Multiply
        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    }

    // Store accumulator fragment back to global memory
    wmma::store_matrix_sync(C + (blockIdx.y * 16 * N) + (blockIdx.x * 16), c_frag, N, wmma::);
}
```

Part III: Attention & Profiling

Profiling & Debugging Like a Pro

Profiling & Debugging Like a Pro

Optimization without profiling is speculation. Top kernel engineers diagnose bottlenecks systematically using the NVIDIA toolchain:

The Professional Diagnostic Stack

1. **compute-sanitizer:** Traps out-of-bounds memory accesses, misaligned memory addresses, uninitialized shared memory, and warp synchronization race conditions.
2. **NVIDIA Nsight Systems (NSYS):** Timeline visualization across CPU threads, CUDA streams, memory transfers, and kernel durations.
3. **NVIDIA Nsight Compute (NCU):** Deep architectural profiling inside individual kernels—analyzing warp stall reasons, SM issue pipe utilization, memory chart traffic, and register spills.

Debugging Sabotaged Kernels: 3 Classic Incident Scenarios

- **Incident 1: Silent Race Condition in Shared Memory:** Threads write to `s_data[threadIdx.x]` and read from `s_data[threadIdx.x + 1]` without an intervening `__syncthreads()`. *Detection:* `compute-sanitizer --tool racecheck ./my_kernel`
- **Incident 2: Shared Memory Bank Conflicts:** All 32 threads in a warp access different rows in the same shared memory bank simultaneously. *Detection:* NCU metric `l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum`. *Resolution:* Stride padding (`__shared__ float smem[32][33]`).
- **Incident 3: Catastrophic Register Spilling:** Kernel uses 80 registers per thread when the hardware limit per thread at target occupancy is 64. Excessive variables spill to local DRAM memory, increasing latency 100x. *Detection:* Compiler flag `--ptxas-options=-v` reporting `spill stores / spill loads`. *Resolution:* Kernel launch bound annotation `__launch_bounds__(256, 2)`.

Attention: The Kernel That Ate the World & FlashAttention v1

Attention: The Kernel That Ate the World & FlashAttention v1

Standard Transformer Attention computes:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

For a sequence length N and head dimension d ($N \gg d$): - $S = QK^T$ generates an $N \times N$ attention score matrix. - Traditional PyTorch implementations materialize S and $P = \text{Softmax}(S)$ in global HBM memory ($O(N^2)$ memory reads and writes). - For long contexts ($N = 32k$ or $128k$), intermediate matrices consume gigabytes of memory and choke HBM bandwidth.

The Breakthrough: Online Softmax & Tiling

Tri Dao et al. introduced **FlashAttention**: compute the exact attention output without ever writing the $N \times N$ intermediate attention matrix to global memory.

Traditional Attention (Memory-Bound):

Q, K -> [Global HBM Write: S] -> [Global HBM Read: S] -> Softmax -> [Global HBM Write: P]

FlashAttention (Fused into SRAM):

Load Blocks of Q, K, V into On-chip SRAM -> Compute Local Gemm -> Online Softmax Update ->

The Online Softmax Formula

Traditional softmax requires two passes over the sequence: pass 1 to find the maximum $m = \max(x_i)$, and pass 2 to compute $\sum e^{x_i - m}$. Online softmax updates the running maximum m and normalization sum l incrementally as blocks stream through shared memory:

$$m_{\text{new}} = \max(m_{\text{prev}}, m_{\text{block}})$$

$$l_{\text{new}} = e^{m_{\text{prev}} - m_{\text{new}}} l_{\text{prev}} + e^{m_{\text{block}} - m_{\text{new}}} l_{\text{block}}$$

$$O_{\text{new}} = e^{m_{\text{prev}} - m_{\text{new}}} O_{\text{prev}} + P_{\text{block}} V_{\text{block}}$$

Part IV: The Modern Frontier

FlashAttention from Scratch (FA1 $\rightarrow$ FA2 $\rightarrow$ FA3)

FlashAttention from Scratch (FA1 → FA2 → FA3)

FlashAttention Evolution

- **FlashAttention-1 (2022):** Proved that fusing attention and computing online softmax in SRAM yields 2-4x wall-clock speedups while cutting memory footprint from $O(N^2)$ to $O(N)$.
- **FlashAttention-2 (2023):**
 - Swapped the outer and inner loops (looping over blocks of Q in the outer loop, streaming K, V in the inner loop).
 - Reduced non-matmul FLOPs by deferring output scaling until the very end.
 - Introduced sequence parallel work distribution across thread blocks.
- **FlashAttention-3 (2024):**
 - Architected specifically for NVIDIA Hopper (H100) hardware.
 - Utilizes asynchronous Tensor Memory Accelerator (TMA) hardware pipelines.
 - Leverages Hopper WGMMMA (Warp Group Matrix Multiply and Accumulate) instructions.
 - Implements FP8 mixed precision with low quantization error.

Beating cuBLAS on an H100

Beating cuBLAS on an H100

NVIDIA's Hopper H100 microarchitecture introduced two revolutionary hardware subsystems:

1. Tensor Memory Accelerator (TMA)

In pre-Hopper architectures, threads in a warp explicitly load data from global memory into registers, then write from registers into shared memory. TMA executes multidimensional tensor transfers (e.g. 2D/3D tiles) directly between global HBM and shared memory **asynchronously in hardware**, completely bypassing the register file and consuming zero SM ALU cycles.

2. Warp Specialization

Instead of every thread in a block performing both memory loading and arithmetic: - **Producer Warps (1-2 warps)**: Issue asynchronous TMA copy instructions to fetch upcoming tiles into double-buffered shared memory. - **Consumer Warps (Warp Groups)**: Execute continuous WGMMMA matrix multiply instructions without ever pausing for memory latency. - Hardware barrier arrive/wait primitives (`cuda::barrier`) synchronize producers and consumers with cycle-level precision.

Triton → CUTLASS → CuTe-DSL

Triton → CUTLASS → CuTe-DSL

Kernel engineering in 2025-2026 has transitioned from hand-written CUDA C++ toward declarative and high-level DSL frameworks.

OpenAI Triton

Triton allows writing high-performance GPU kernels in idiomatic Python. The programmer writes code at the granularity of block tiles rather than individual threads. The Triton compiler handles intra-block scheduling, memory coalescing, and shared memory allocation automatically.

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr, y_ptr, output_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(output_ptr + offsets, x + y, mask=mask)
```

CUTLASS & CuTe

NVIDIA CUTLASS 3.x is structured around **CuTe**: a C++ template library that unifies data layouts, coordinates, and hierarchical tensor tiling through mathematical layout algebra:

$$extLayout = (extShape, extStride)$$

Inference-Serving Kernels

Inference-Serving Kernels

LLM inference splits into two distinct execution regimes with radically different performance characteristics: 1. **Prefill Phase:** The prompt is processed in parallel. High operational intensity, strictly compute-bound. 2. **Decode Phase:** Tokens are generated autoregressively one by one. The model loads billions of weights to compute a single new vector. Strictly memory-bound!

PagedAttention (vLLM)

In standard inference, KV caches must be allocated as contiguous memory buffers. This leads to 60-80% memory waste due to internal and external memory fragmentation. PagedAttention treats KV cache memory like virtual memory pages in an operating system: - KV vectors are stored in non-contiguous 16-token page blocks. - A page table maps logical token positions to physical GPU memory addresses. - Result: Memory waste drops to near 0%, enabling 2-4x higher serving concurrency on the same hardware.

Blackwell & NVFP4

Blackwell & NVFP4

NVIDIA's Blackwell (B200 / GB200) microarchitecture represents the state of the art in frontier hardware (2025-2026).

Blackwell Architectural Highlights

- **tcgen05 Tensor Cores:** 5th generation Tensor Cores delivering 4x higher FP4 training and inference density compared to Hopper FP8.
- **Tensor Memory (TMEM):** A dedicated ultra-fast on-chip memory structure decoupled from shared memory, specifically optimized for multi-tensor staging and GEMM accumulation.
- **NVFP4 Microscopic Scaling:** 4-bit floating point format with micro-block scaling factors that preserves accuracy across large foundation models while doubling token throughput.

Flash Attention 4

Flash Attention 4

The 2026 Frontier: Breaking the 1 PFLOPS Single-GPU Barrier

On Blackwell hardware, Tensor Core matrix multiply units run so extraordinarily fast that the normalization step in Softmax—historically a minor fraction of runtime—becomes the dominant bottleneck.

FlashAttention-4 introduces **Adaptive Softmax**: - Dynamically checks divergence between running block maximums and candidate maximums. - Skips full re-normalization when numerical differences fall below precision thresholds. - Overlaps Softmax correction entirely behind asynchronous WGMMMA execution pipelines. - Written using modern Python CuTe-DSL, FA4 is the first kernel to achieve over 1 PFLOPS of sustained attention compute on a single processor.

PagedAttention and the KV Cache

PagedAttention and the KV Cache

Autoregressive LLM serving is memory-bound in decode and **capacity-bound** in concurrency: each active sequence carries a growing key/value (KV) cache. Contiguous per-sequence KV allocations waste large fractions of GPU DRAM to fragmentation and over-reservation. **PagedAttention** (vLLM) borrows the OS virtual-memory idea—fixed-size blocks, a block table, allocate-on-demand—so attention kernels read logically contiguous tokens from physically non-contiguous KV blocks.

This chapter goes deeper than a one-paragraph mention in inference-serving notes: why contiguous KV fails, how block tables work, prefill vs decode implications, layout choices for custom kernels, and how to verify/benchmark paging behavior.

Mental model

OS analogy	LLM serving
Process virtual address space	Logical token positions $0..T - 1$ in a sequence
Pages	KV blocks of B tokens
Page table	Per-sequence block table (logical block $\rightarrow$ physical block)
Physical frames	Slots in a GPU KV pool
Internal fragmentation	At most one partially filled tail block
External fragmentation	Largely eliminated (any free block can be used)

```
logical tokens:  [t0 t1 ... t15 | t16 ... t31 | t32 ... t40____]
                  block 0          block 1          block 2 (partial)
```

```
block table:    L0  $\rightarrow$  P17
                 L1  $\rightarrow$  P3
                 L2  $\rightarrow$  P9
```

```
physical pool:  P0 P1 P2 P3[K/V] ... P9[K/V] ... P17[K/V] ...
```

Attention still needs Q against all prior K , and weighted sum over V —but the kernel **gathers** K, V through the block table instead of assuming one contiguous pointer.

Why contiguous KV wastes memory

For a sequence with maximum length $L_{\max}$ and current length t :

1. **Reservation waste:** many systems allocate KV for $L_{\max}$ up front even while $t \ll L_{\max}$.
2. **External fragmentation:** variable-length contiguous regions cannot pack tightly as sequences finish at different times.
3. **Batching rigidity:** contiguous layouts make sharing prefixes or compacting live sequences expensive.

Empirical reports in the PagedAttention / vLLM line of work put waste often in the **60–80%** range under contiguous reservation—memory that cannot become additional concurrent sequences. Paging caps internal waste at roughly one block (B tokens) per sequence (per managed cache partition) and removes most external fragmentation.

Capacity sketch

Let one token’s KV footprint (all layers, all heads, K and V) be b bytes. Contiguous reservation for N sequences costs $\sim NL_{\max}b$. Paged on-demand costs $\sim b \sum_i t_i$ plus $< NBb$ partial-block slack (order-of-magnitude). The ratio $L_{\max}/\bar{t}$ dominates the win when outputs are much shorter than the configured max.

KV cache in Transformers (reminder)

For each layer ℓ and head h , token position i stores vectors $k_i^{(\ell,h)}, v_i^{(\ell,h)} \in \mathbb{R}^d$. Causality at decode step t uses

$$\alpha_{t,i} = \text{softmax}_i \left(\frac{q_t^\top k_i}{\sqrt{d}} \right), \quad o_t = \sum_{i=1}^t \alpha_{t,i} v_i.$$

Prefill computes many q rows in parallel against the prompt’s K, V ; decode appends one new (k_t, v_t) and attends with a single q_t (or a small speculative set).

Serving reality: weights are shared; **KV grows per request**. Throughput (tokens/s/GPU) often rises when more sequences fit—hence KV packing is a first-class systems problem, not a minor allocator tweak.

Block structure

Choose block size B (vLLM commonly discusses values like 16). Logical block j holds tokens $(j-1)B+1, \dots, jB$ (or 0-based equivalents):

$$K_j = (k_{(j-1)B+1}, \dots, k_{jB}), \quad V_j = (v_{(j-1)B+1}, \dots, v_{jB}).$$

Attention becomes a loop over logical blocks: load the physical block pointed by the table, compute partial scores/outputs, advance. The last block carries a **filled-token count** $1..B$.

Worked layout numbers

$B = 16$, head dim $d = 128$, dtype FP16: one head's K block stores 16×128 elements = 2048 halves ≈ 4 KiB (order of magnitude), and V similarly. Multi-layer multi-head caches scale linearly; production engines may use separate block tables per layer/head partition or pack carefully for coalescing—design choice, not a change to the paging idea.

Block table and allocator

Per sequence (and often per KV partition):

Field	Role
Logical index j	Token range $[jB, (j+1)B)$
Physical block id	Index into GPU (or CPU swap) pool
Filled count	Valid tokens in the tail block
Refcount (advanced)	Prefix sharing / copy-on-write

Allocate on demand: when decode would write past the filled count, pull a free physical block, append a table entry. **Free on finish:** return blocks to the pool (and decrement refcounts if shared).

```
decode step:
  if filled == B:
    phys = alloc_block()
    table.append(phys)
    filled = 0
  write k_t, v_t into table[-1] at offset filled
  filled += 1
  run paged_attention(q_t, table)
```

Prefix sharing (copy-on-write intuition)

If two requests share a prompt prefix, map their early logical blocks to the **same** physical blocks with elevated refcounts. Divergence (different generated tokens) allocates private blocks from the fork point. Automatic prefix caching builds on the same indirection: hash logical block contents (+prefix identity) to reuse physical KV.

Prefill vs decode under paging

Phase	Compute character	Paging interaction
Prefill	Compute-heavy; many tokens written	Allocate enough blocks for prompt; write K/V densely left-to-right; attention over prompt blocks
Decode	Memory-bound; one token at a time	Append into tail block; attention gathers all prior blocks for one q

Custom kernels must handle **variable block lists per sequence in a batch**: sequence A may have 3 blocks, sequence B 40. Batching strategies (continuous batching) reshape which sequences sit in a wave while the block tables remain the source of truth for KV addresses.

Implication

A FlashAttention-style tiled softmax over a **contiguous** K, V buffer is not drop-in compatible. You need a kernel that:

1. Accepts block tables + pool base pointers, or
2. Materializes contiguous KV (defeats paging), or
3. Uses a hybrid (e.g., paged decode kernels + specialized prefill paths).

vLLM’s historical PagedAttention CUDA kernels (`attention_kernels`-style designs) are built for case (1): thread blocks iterate paged K/V vectors with careful vectorized loads.

Paged KV layouts (kernel writer view)

Exact layouts evolve; treat the following as a **mental checklist** when reading kernels or writing prototypes:

```
Pool: physical_block[p] → storage for B tokens × head_dim (K or V)
```

```
K access for logical token t:
```

```
  j = t / B
```

```
  o = t % B
```

```
  p = block_table[seq][j]
```

```
  addr = base_K + p * stride_block + o * stride_token + head_offset
```

```
V access: similar with base_V / possibly different vectorization shape
```

K vs V vectorization: kernels often pack K for efficient $Q \cdot K$ dots (e.g., threads cooperating on a head vector) while V layouts favor weighting and accumulating into the output. Do not assume K and V share identical lane mapping.

Separate K and V caches: common—two pools, parallel tables or mirrored indices.

Illustrative host-side table (not a production kernel)

```
# file: toy_block_table.py
# Toy model of logical→physical mapping (host-side). Not CUDA.

from dataclasses import dataclass, field

B = 16

@dataclass
class SequenceKV:
    table: list[int] = field(default_factory=list) # physical ids
    filled: int = 0 # tokens used in last block

class BlockPool:
    def __init__(self, n_blocks: int):
        self.free = list(range(n_blocks))

    def alloc(self) -> int:
        if not self.free:
            raise MemoryError("KV pool exhausted")
        return self.free.pop()

    def free_block(self, p: int) -> None:
        self.free.append(p)

def append_token(seq: SequenceKV, pool: BlockPool) -> None:
    if not seq.table or seq.filled == B:
        seq.table.append(pool.alloc())
        seq.filled = 0
    # device would write K/V into pool[seq.table[-1]][seq.filled]
    seq.filled += 1

def release(seq: SequenceKV, pool: BlockPool) -> None:
    for p in seq.table:
        pool.free_block(p)
    seq.table.clear()
    seq.filled = 0
```

Use this only to reason about allocation traffic; real engines track layers/heads, CUDA streams, and fragmentation across devices.

Implications for custom kernels

1. **Indirection tax:** every K/V load pays a table lookup (often cached in shared memory / registers for the active block list).
2. **Tail blocks:** masks for `filled < B` are mandatory—same family of bugs as ragged sequence lengths.
3. **Batching:** one wave may include many tables; prefer grouped launches or paged-friendly attention APIs over naively padding to $L_{\max}$ contiguous.
4. **Fusion:** paging composes with continuous batching, speculative decoding (extra parallel tokens), and quantization of KV (INT8/FP8 caches)—each changes strides and scales, not the block-table story.
5. **Swap / multi-tier:** physical blocks may live in CPU RAM; kernels and schedulers must not assume all table entries are GPU-resident without a residency check.

When generating kernels with AI assistants: specify **block size**, **K/V separation**, **dtype**, **head dim**, and **tail masking** explicitly. Ambiguous “write attention” prompts usually emit contiguous FlashAttention clones that cannot consume a paged pool.

Verification and benchmark notes

Functional checks

Check	Idea
Match contiguous golden	For small T , gather paged KV to contiguous and compare attention outs vs a reference softmax
Tail mask	Lengths $B - 1$, B , $B + 1$ across block boundaries
Multi-sequence batch	Different table lengths in one launch
Prefix share	RefCount: mutate after fork must not clobber peer

Performance / capacity checks

Metric	Why
Max concurrent sequences at fixed $L_{\max}$	Primary paging win
KV bytes reserved vs bytes touched	Fragmentation / over-reserve detector

Metric	Why
Decode tokens/s vs batch size	Memory bandwidth + kernel efficiency
Allocation failures under load	Pool sizing / preemption / swap policy

Microbenchmarks that only time a contiguous GEMM miss the point—page-table attention is about **fitting more live KV** without catastrophic gather overhead. Profile both **allocator** and **attention kernel**.

Profiling tips

- Trace block alloc/free counts per request; spikes may indicate missing prefix sharing.
- Compare achieved occupancy / bytes loaded for paged vs contiguous decode on identical batch shapes.
- Watch for CPU-sync on block-table uploads each step—pin tables or update incrementally.

Pitfalls

Pitfall	Result
Assuming contiguous FlashAttention works unchanged	Wrong answers or illegal loads
Ignoring partial-block fill	Garbage tokens in softmax
Oversized B	More internal waste; less flexible packing
Undersized pool	Preemption thrash or request rejects
Forgetting refcount on shared prefixes	Silent cross-request corruption
Benchmarking only prefill TFLOPs	Misses decode memory + paging behavior

Checkpoint

- Explain why contiguous $L_{\max}$ reservation destroys concurrency.
- Map OS paging terms onto block tables and KV pools.
- Describe allocate-on-demand during decode and tail **filled** counts.
- List what a paged attention kernel must accept beyond Q, K, V pointers.
- Name two functional tests and one capacity metric for paging.

Summary

PagedAttention makes KV cache **virtually contiguous and physically sparse**. The win is utilization—more sequences per GPU—at the cost of attention kernels that understand block tables, tail masks, and pool lifetimes. Prefill still wants heavy compute paths; decode wants lean paged gathers. For custom or AI-written kernels, treat the block table as part of the ABI, verify against contiguous goldens at block boundaries, and benchmark concurrency and KV bytes—not only matmul TFLOPs.

Sources (for further reading)

- Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention” (SOSP 2023; arXiv:2309.06180).
- vLLM design docs on PagedAttention / paged KV layouts.
- vLLM blog: “Easy, Fast, and Cheap LLM Serving with PagedAttention.”

Flash-Decoding: Split-KV Attention for Long-Context Decode

Flash-Decoding: Split-KV Attention for Long-Context Decode

FlashAttention made **prefill** fast by tiling $QK^\top$, softmax, and PV into one kernel that never writes the score matrix to HBM. **Decode** is a different problem. Each step has *one* new query row per sequence and head, attending over a KV cache that may hold tens or hundreds of thousands of tokens. A FlashAttention-style grid parallelizes over $\text{batch} \times \text{heads} \times \text{query tiles}$, and at decode there is only one query tile. With a small batch and a long context, that leaves a handful of CTAs to stream gigabytes of KV while most SMs sit idle.

Flash-Decoding (Dao, Haziza, Massa, Sizov, 2023) adds the missing parallel dimension: the **KV sequence**. Each split of the cache runs the familiar online-softmax loop independently and emits a partial output plus one scalar, its **log-sum-exp (LSE)**. A tiny second kernel then merges the splits exactly. The authors reported up to $8\times$ faster decode for very long sequences, and the same split-then-merge pattern now sits under paged decode kernels, cascade or shared-prefix attention, and context-parallel attention across GPUs.

This chapter builds the idea from the algebra up. It proves the merge is exact, writes a two-kernel Triton implementation that you can verify on a laptop with the Triton interpreter, compiles it for a GPU you do not own, ports the production split-count heuristic, and lists the checks that catch the real bugs.

1. Why decode starves the GPU

For one (sequence, head) pair at decode step t :

$$s_i = \frac{q^\top k_i}{\sqrt{d}}, \quad o = \sum_{i=1}^t \text{softmax}(s)_i v_i.$$

The work is about $4td$ FLOPs against $2td$ elements of K and V that must be read from HBM. In FP16 that is about **1 FLOP per byte**, which puts it far left on the roofline: decode attention is purely **bandwidth-bound**. The only goal is to keep every SM's load pipeline full.

Kernel style	Grid at decode	CTAs for batch 1, 8 KV heads
FlashAttention (one CTA per batch $\times$ head $\times$ query tile)	$B \cdot H \cdot 1$	8

Kernel style	Grid at decode	CTAs for batch 1, 8 KV heads
Flash-Decoding (adds S KV splits)	$B \cdot H \cdot S$	$8S$ (e.g. 232 with $S = 29$)

Worked numbers

Batch 1, 8 KV heads (grouped-query attention), context 128 000, head dim 128, FP16 cache. K plus V for **one layer** is

$$2 \times 8 \times 128\,000 \times 128 \times 2 \text{ B} \approx 524 \text{ MB.}$$

At the H100 SXM’s roughly 3.35 TB/s HBM3 peak, that is about 160 μs per layer if bandwidth is saturated. Eight CTAs on a 132-SM part cannot get close: each SM can only keep a limited number of loads in flight. Splitting the sequence 29 ways gives 232 CTAs, enough to occupy every SM and keep loads outstanding. The model is the same and so are the bytes; the kernel is simply parallelized along the dimension that actually has work.

GQA note: with grouped-query attention, several query heads share one KV head. Good decode kernels pack the query heads of a group into the “M” rows of a tile (FlashAttention-3 calls this `pack_gqa`), so the KV block is loaded once per group, not once per query head. The parallelism problem, and the split-KV fix, are about **KV heads** $\times$ **batch**.

2. Mental model: attention states and the merge operator

For a set of key indices I , define its **attention state** as the pair

$$\text{LSE}(I) = \log \sum_{i \in I} e^{s_i}, \quad o(I) = \sum_{i \in I} \frac{e^{s_i}}{e^{\text{LSE}(I)}} v_i.$$

$o(I)$ is the attention output you would get if the cache contained **only** the keys in I . $\text{LSE}(I)$ records how much total softmax mass I carries.

Claim. For disjoint I and J :

$$\text{LSE}(I \cup J) = \log(e^{\text{LSE}(I)} + e^{\text{LSE}(J)}), \quad o(I \cup J) = \frac{e^{\text{LSE}(I)} o(I) + e^{\text{LSE}(J)} o(J)}{e^{\text{LSE}(I)} + e^{\text{LSE}(J)}}.$$

Proof. $e^{\text{LSE}(I)} o(I) = \sum_{i \in I} e^{s_i} v_i$ is the *unnormalized* numerator over I , and $e^{\text{LSE}(I)}$ is the unnormalized denominator. Numerators and denominators over disjoint sets simply add. ■

Call this merge $\oplus$. It is **associative and commutative**, so the splits can be combined in any order: sequentially, as a tree, across warps, across CTAs, or across GPUs. It is the same operator FlashAttention applies tile by tile inside one CTA (the online softmax), now applied *between* CTAs.

FlashInfer makes it the canonical output of every attention kernel (“attention state”) and exposes `merge_state` / `merge_states` as first-class operations.

Numerically stable form. Never exponentiate an LSE directly. Subtract the maximum first:

$$m = \max_j \text{LSE}_j, \quad w_j = e^{\text{LSE}_j - m}, \quad o = \frac{\sum_j w_j o_j}{\sum_j w_j}, \quad \text{LSE} = m + \log \sum_j w_j.$$

```

KV cache for one (sequence, head):  t tokens

split 0          split 1          split 2          split 3          (last split may be short or empty)

online softmax over BLOCK_N tiles (kernel 1, one CTA per split)

(o0, lse0)        (o1, lse1)        (o2, lse2)        (o3, lse3)        fp32 workspace in HBM

                                kernel 2: m = max lse; w = exp(lse - m); o = Σ w·o / Σ w
                                final o  (and lse, if a caller wants to merge further)

```

3. Verify the algebra before writing a kernel

Start with a host reference so that every GPU result later has a ground truth.

```

# file: split_kv_reference.py
# NumPy reference: one decode query against a long KV cache, computed
# (a) in one pass, and (b) as Flash-Decoding style splits merged by log-sum-exp.
import numpy as np

rng = np.random.default_rng(0)
D, T, S = 128, 10_000, 7 # head dim, cached tokens, number of KV splits
q = rng.standard_normal(D).astype(np.float32)
K = rng.standard_normal((T, D)).astype(np.float32)
V = rng.standard_normal((T, D)).astype(np.float32)
scale = 1.0 / np.sqrt(D)

def attend(q, K, V):
    """Return (o, lse) for one query over a KV segment: o normalised, lse = log sum exp(s)."""
    s = (K @ q) * scale
    m = s.max()
    p = np.exp(s - m)
    l = p.sum()
    return (p @ V) / l, m + np.log(l)

```

```

def merge(states):
    """Combine (o_i, lse_i) partial states into the exact full-segment state."""
    lse = np.array([st[1] for st in states])
    m = lse.max()
    w = np.exp(lse - m) # unnormalised weight of each split
    o = sum(wi * st[0] for wi, st in zip(w, states)) / w.sum()
    return o, m + np.log(w.sum())

o_full, lse_full = attend(q, K, V)

bounds = np.linspace(0, T, S + 1).astype(int) # uneven split lengths are fine
parts = [attend(q, K[a:b], V[a:b]) for a, b in zip(bounds[:-1], bounds[1:])]
o_split, lse_split = merge(parts)

# The merge is associative: merging pairwise in a tree gives the same answer.
tree = merge([merge(parts[:3]), merge(parts[3:])])

print("split boundaries :", bounds.tolist())
print("max |o_full - o_split| =", float(np.abs(o_full - o_split).max()))
print("|lse_full - lse_split| =", float(abs(lse_full - lse_split)))
print("max |o_split - o_tree| =", float(np.abs(o_split - tree[0]).max()))

python3 -m venv .venv && . .venv/bin/activate
pip install numpy
python split_kv_reference.py

```

Output:

```

split boundaries : [0, 1428, 2857, 4285, 5714, 7142, 8571, 10000]
max |o_full - o_split| = 3.8762266595278316e-10
|lse_full - lse_split| = 1.7920598338605487e-10
max |o_split - o_tree| = 1.3877787807814457e-17

```

Split, merge, or tree-merge: the results agree to rounding. Uneven split lengths do not matter, because each split's LSE carries its own weight.

The trap: the textbook merge overflows

```

# file: merge_trap.py
# The textbook merge o = sum(exp(lse_i) * o_i) / sum(exp(lse_i)) overflows float32
# as soon as any lse_i > ~88.7. Subtracting max(lse) first is not optional.
import numpy as np

```

```

lse = np.array([95.0, 97.5, 91.2], dtype=np.float32)          # plausible with large logits
o = np.array([[1.0, 0.0], [0.0, 1.0], [0.5, 0.5]], dtype=np.float32)

with np.errstate(over="ignore", invalid="ignore"):
    w_naive = np.exp(lse)
    print("naive weights :", w_naive)
    print("naive merge   :", (w_naive[:, None] * o).sum(0) / w_naive.sum())

m = lse.max()
w = np.exp(lse - m)
print("stable weights:", w)
print("stable merge   :", (w[:, None] * o).sum(0) / w.sum())
print("merged lse     :", m + np.log(w.sum()))

```

```
python merge_trap.py
```

Output:

```

naive weights : [inf inf inf]
naive merge   : [nan nan]
stable weights: [0.08208501 1.          0.0018363 ]
stable merge   : [0.07657673 0.92342323]
merged lse     : 97.58058

```

exp overflows FP32 above about 88.7, and LSE values that large are routine once logits grow (long contexts, no QK-norm, large activations). Kernels that “pass the unit test” with `randn` inputs and then NaN in production usually have exactly this bug in the combine step.

4. A two-kernel Triton implementation

The design is the minimum that is still honest about production concerns:

- **Kernel 1** (`split_kv_kernel`), grid $(B \cdot H, S)$. Each program owns one split of one (sequence, head) pair, streams its keys in `BLOCK_N` tiles with the online-softmax recurrence, and writes a normalized partial output and its LSE to an FP32 workspace.
- **Kernel 2** (`combine_kernel`), grid $(B \cdot H)$. Loads S LSEs and S partial outputs, applies the stable merge, and writes the result.
- **Ragged batches.** `SEQ_LENS[b]` bounds every sequence, so a sequence shorter than the split layout leaves some splits **empty**. An empty split must write $o = 0$ and $\text{LSE} = -\infty$, giving it weight $e^{-\infty} = 0$. Writing $0/0$ there is the classic NaN.
- **FP32 everywhere it matters.** Scores, running max, running sum, accumulator, and workspace are all FP32, whatever the cache dtype. FlashAttention-2’s split-KV path allocates its `out_accum` and `softmax_lse_accum` workspaces in FP32 for the same reason.

```

# file: flash_decode_triton.py
# Two-kernel Flash-Decoding (split-KV) for batched single-token decode, written in Triton.
#   kernel 1: grid = (batch*heads, num_splits) -> partial (o, lse) per split
#   kernel 2: grid = (batch*heads,) -> log-sum-exp merge across splits
# Runs on a GPU, or on CPU for correctness work with: TRITON_INTERPRET=1 python flash_decod
import math
import os

import torch
import triton
import triton.language as tl

@triton.jit
def split_kv_kernel(Q, K, V, SEQ_LENS, O_PART, LSE_PART,
                   stride_kb, stride_kh, stride_kt,      # K/V: [B, H, T_max, D], last dim
                   H, chunk, scale,
                   D: tl.constexpr, BLOCK_N: tl.constexpr, NUM_SPLITS: tl.constexpr):
    bh = tl.program_id(0)
    split = tl.program_id(1)
    b = bh // H
    h = bh % H
    seq_len = tl.load(SEQ_LENS + b)

    start = split * chunk
    end = tl.minimum(start + chunk, seq_len)           # may be <= start: empty split

    offs_d = tl.arange(0, D)
    q = tl.load(Q + bh * D + offs_d).to(tl.float32)   # Q: [B*H, D]
    kv_base = b * stride_kb + h * stride_kh

    m_i = -float("inf")                                # running max of scores
    l_i = 0.0                                           # running sum of exp(score - m_i)
    acc = tl.zeros([D], dtype=tl.float32)             # running sum of p * v

    for n0 in range(start, end, BLOCK_N):
        offs_n = n0 + tl.arange(0, BLOCK_N)
        mask_n = offs_n < end
        ptrs = kv_base + offs_n[:, None] * stride_kt + offs_d[None, :]
        k = tl.load(K + ptrs, mask=mask_n[:, None], other=0.0).to(tl.float32)
        s = tl.sum(k * q[None, :], axis=1) * scale      # [BLOCK_N] scores
        s = tl.where(mask_n, s, -float("inf"))
        m_new = tl.maximum(m_i, tl.max(s, axis=0))
        alpha = tl.exp(m_i - m_new)                    # rescale old state
        p = tl.exp(s - m_new)
        v = tl.load(V + ptrs, mask=mask_n[:, None], other=0.0).to(tl.float32)
        acc = acc * alpha + tl.sum(p[:, None] * v, axis=0)

```

```

        l_i = l_i * alpha + tl.sum(p, axis=0)
        m_i = m_new

    has_keys = end > start
    safe_l = tl.where(has_keys, l_i, 1.0)
    o = tl.where(has_keys, acc / safe_l, 0.0) # empty split -> o = 0 ...
    lse = tl.where(has_keys, m_i + tl.log(safe_l), -float("inf")) # ... with weight exp(-in
    out = bh * NUM_SPLITS + split
    tl.store(O_PART + out * D + offs_d, o)
    tl.store(LSE_PART + out, lse)

@triton.jit
def combine_kernel(O_PART, LSE_PART, 0,
                  D: tl.constexpr, NUM_SPLITS: tl.constexpr):
    bh = tl.program_id(0)
    offs_s = tl.arange(0, NUM_SPLITS)
    offs_d = tl.arange(0, D)
    lse = tl.load(LSE_PART + bh * NUM_SPLITS + offs_s) # [S]
    m = tl.max(lse, axis=0)
    w = tl.exp(lse - m) # empty splits contribute 0
    o_part = tl.load(O_PART + (bh * NUM_SPLITS + offs_s)[: , None] * D + offs_d[None, :])
    o = tl.sum(w[: , None] * o_part, axis=0) / tl.sum(w, axis=0)
    tl.store(O + bh * D + offs_d, o)

def flash_decode(q, k, v, seq_lens, num_splits=8, block_n=32):
    """q: [B, H, D]; k, v: [B, H, T_max, D]; seq_lens: [B] int32. Returns [B, H, D] float32.
    B, H, T_max, D = k.shape
    chunk = triton.cdiv(T_max, num_splits)
    o_part = torch.empty(B * H, num_splits, D, dtype=torch.float32, device=q.device)
    lse_part = torch.empty(B * H, num_splits, dtype=torch.float32, device=q.device)
    out = torch.empty(B, H, D, dtype=torch.float32, device=q.device)
    split_kv_kernel[(B * H, num_splits)](
        q.contiguous(), k, v, seq_lens, o_part, lse_part,
        k.stride(0), k.stride(1), k.stride(2),
        H, chunk, 1.0 / math.sqrt(D),
        D=D, BLOCK_N=block_n, NUM_SPLITS=num_splits)
    combine_kernel[(B * H,)](o_part, lse_part, out, D=D, NUM_SPLITS=num_splits)
    return out

def reference(q, k, v, seq_lens):
    B, H, _, D = k.shape
    out = torch.empty(B, H, D, dtype=torch.float32)
    for b in range(B):
        L = int(seq_lens[b])

```

```

        s = torch.einsum("hd,htd->ht", q[b].float(), k[b, :, :L].float()) / math.sqrt(D)
        out[b] = torch.einsum("ht,htd->hd", torch.softmax(s, dim=-1), v[b, :, :L].float())
    return out

if __name__ == "__main__":
    device = "cpu" if os.environ.get("TRITON_INTERPRET") == "1" else "cuda"
    torch.manual_seed(0)
    B, H, T_max, D = 3, 4, 1000, 64
    seq_lens = torch.tensor([1000, 37, 1], dtype=torch.int32, device=device) # ragged batch
    q = torch.randn(B, H, D, device=device)
    k = torch.randn(B, H, T_max, D, device=device)
    v = torch.randn(B, H, T_max, D, device=device)
    for q_gain in (1.0, 40.0):
        # 40x -> logits in the hundreds
        for splits in (1, 4, 8):
            qq = q * q_gain
            got = flash_decode(qq, k, v, seq_lens, num_splits=splits)
            ref = reference(qq.cpu(), k.cpu(), v.cpu(), seq_lens.cpu())
            err = (got.cpu() - ref).abs().max().item()
            print(f"q_gain={q_gain:>4}  num_splits={splits}:  max abs error = {err:.2e}")

```

Run it on a CPU with the Triton interpreter. The same file runs unchanged on a CUDA GPU without the environment variable.

```

pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install triton
TRITON_INTERPRET=1 python flash_decode_triton.py

```

Output (Triton 3.8.0, PyTorch 2.14.1 CPU, interpreter mode):

```

q_gain= 1.0  num_splits=1:  max abs error = 2.38e-07
q_gain= 1.0  num_splits=4:  max abs error = 2.38e-07
q_gain= 1.0  num_splits=8:  max abs error = 2.38e-07
q_gain=40.0  num_splits=1:  max abs error = 9.18e-06
q_gain=40.0  num_splits=4:  max abs error = 9.06e-06
q_gain=40.0  num_splits=8:  max abs error = 9.06e-06

```

What the test matrix covers:

Case	Why
num_splits=1	Degenerates to plain FlashAttention decode: the baseline
seq_lens = [1000, 37, 1]	Ragged batch. With 8 splits of 125 tokens, sequence 1 has 7 empty splits and sequence 2 has a one-token cache
q_gain=40	Logits in the hundreds: exercises the max-subtraction in both kernels

Case	Why
Comparison against <code>torch.softmax</code>	An independent reference, not a second copy of the same algorithm

The interpreter executes the kernel's tile semantics with NumPy on the CPU. It proves **correctness**, not performance: it does not model warps, shared memory, or bandwidth. It is still the fastest loop for an AI-assisted workflow, because a generated kernel can be run against a reference in seconds without a GPU.

5. Compile for a GPU you do not have

Interpreter success does not guarantee that the kernel lowers. Loop-carried values that change dtype, non-constexpr shapes, and unsupported ops fail only in the real compiler. Triton can compile ahead of time for a named target on a CPU-only machine:

```
# file: compile_check.py
# Ahead-of-time compile both kernels for an H100 (sm_90) on a machine with no GPU.
# Catches type errors, bad constexpr use and loop-carried dtype mismatches before you rent a
import triton
from triton.backends.compiler import GPUSubtarget
from triton.compiler import ASTSource

from flash_decode_triton import combine_kernel, split_kv_kernel

target = GPUSubtarget("cuda", 90, 32) # backend, compute capability, warp size
split_sig = {"Q": "*fp32", "K": "*fp32", "V": "*fp32", "SEQ_LENS": "*i32",
             "O_PART": "*fp32", "LSE_PART": "*fp32",
             "stride_kb": "i32", "stride_kh": "i32", "stride_kt": "i32",
             "H": "i32", "chunk": "i32", "scale": "fp32",
             "D": "constexpr", "BLOCK_N": "constexpr", "NUM_SPLITS": "constexpr"}
combine_sig = {"O_PART": "*fp32", "LSE_PART": "*fp32", "O": "*fp32",
               "D": "constexpr", "NUM_SPLITS": "constexpr"}

jobs = [
    ("split_kv_kernel", ASTSource(fn=split_kv_kernel, signature=split_sig,
                                constexprs={"D": 128, "BLOCK_N": 64, "NUM_SPLITS": 32})),
    ("combine_kernel", ASTSource(fn=combine_kernel, signature=combine_sig,
                                constexprs={"D": 128, "NUM_SPLITS": 32})),
]
for name, src in jobs:
    k = triton.compile(src, target=target)
    md = k.metadata
    print(f"{name:16s} stages={'.'.join(k.asm.keys())} ")
```

```
f"num_warps={md.num_warps} shared={md.shared} B ptx_lines={k.asm['ptx'].count(ch
```

```
python compile_check.py
```

Output (Triton 3.8.0 on a CPU-only Linux box):

```
split_kv_kernel  stages=source,ttir,ttgir,llir,ptx,cubin  num_warps=4  shared=128 B  ptx_lin
combine_kernel   stages=source,ttir,ttgir,llir,ptx,cubin  num_warps=4  shared=0 B  ptx_lines
```

Both kernels lower all the way to an `sm_90` cubin. `k.asm["ptx"]` and `k.asm["ttgir"]` are there for inspection, and they repay it. In this build every K/V load comes out as a scalar `ld.global.b32`, because the hand-written signature tells the compiler nothing about pointer alignment or stride divisibility. At a normal launch, Triton specializes on arguments that are divisible by 16, and that specialization is what typically unlocks wider vector loads. Grepping the PTX of the kernel you actually launch is a cheap way to confirm you got them.

Wire this compile check, together with the interpreter test, into the agent loop as a cheap gate before any GPU time is spent.

6. How many splits?

Too few splits leave SMs idle. Too many add workspace traffic and combine work, and make each split too short to amortize its prologue. FlashAttention-2's dispatcher chooses S by **wave efficiency**: launch $B \cdot H \cdot S$ CTAs onto a fixed number of slots, and prefer the smallest S whose last wave is nearly as full as the best achievable.

```
# file: choose_splits.py
# Wave-efficiency heuristic for picking the number of KV splits, ported from the
# FlashAttention-2 split-KV dispatcher (num_splits_heuristic in csrc/flash_attn/flash_api.cp
import math

def num_splits(work_items, slots, n_kv_blocks, max_splits=128):
    if work_items >= 0.8 * slots:          # already ~fills the machine: do not split
        return 1
    max_splits = min(max_splits, slots, n_kv_blocks)

    def eligible(s):
        # 11 vs 12 splits of 64 blocks is really 11 splits
        return s == 1 or math.ceil(n_kv_blocks / s) != math.ceil(n_kv_blocks / (s - 1))

    eff = {}
    for s in range(1, max_splits + 1):
        if eligible(s):
            waves = work_items * s / slots
```

```

    eff[s] = waves / math.ceil(waves)    # fraction of the last wave that is busy
    best = max(eff.values())
    return next(s for s in sorted(eff) if eff[s] >= 0.85 * best)

```

```

SMS = 132                                # e.g. H100 SXM; FA2 passes 2 * num_SMS as the slot count
BLOCK_N = 128                            # KV tokens per inner tile for head_dim 128
print(f"{'batch':>5} {'heads':>8} {'context':>8} {'work':>5} {'splits':>6} {'CTAs':>5}")
for batch, heads, ctx in [(1, 8, 128_000), (1, 32, 32_000), (4, 8, 8_000),
                           (16, 8, 4_000), (64, 32, 2_000), (1, 8, 300)]:
    work = batch * heads
    s = num_splits(work, 2 * SMS, math.ceil(ctx / BLOCK_N))
    print(f"{'batch':>5} {'heads':>8} {'ctx':>8} {'work':>5} {'s':>6} {'work * s:>5}")

```

```
python3 choose_splits.py
```

Output:

batch	heads	context	work	splits	CTAs
1	8	128000	8	29	232
1	32	32000	32	7	224
4	8	8000	32	7	224
16	8	4000	128	2	256
64	32	2000	2048	1	2048
1	8	300	8	3	24

Reading the table:

- **Small batch, long context** is where Flash-Decoding matters: 8 work items become 232 CTAs.
- **Large batch** already fills the machine (2048 work items), so the heuristic returns 1 split and the extra pass is skipped entirely.
- **Short context** is capped by the number of KV tiles: 300 tokens are only 3 tiles of 128, so at most 3 splits.
- The “eligible” check skips split counts that produce the same tiles per split as a smaller count, which would add a combine for nothing.

Overhead check. Per (sequence, head), the workspace costs $S \cdot d$ FP32 values written and read back, about $8Sd$ bytes, while the KV stream is $4td$ bytes in FP16. The ratio $2S/t$ is about 0.05% for $S = 29$, $t = 128000$. The combine kernel should be a sliver in an Nsight Systems timeline. If it is not, S is too large or the combine is badly written.

7. Where the pattern shows up in production

System	How it uses split-KV and merge
FlashAttention-2 (2.2), <code>flash_attn_with_kv_cache</code>	<code>num_splits=0</code> selects the heuristic above; <code>block_table=</code> combines splits with a paged KV cache; <code>return_softmax_lse=True</code> returns the state for further merging
FlashAttention-3 (Hopper) interface	Exposes <code>num_splits</code> and <code>pack_gqa</code> for decode-shaped problems
FlashInfer	“Attention state” (<code>o</code> , <code>lse</code>) is the standard output; <code>merge_state</code> / <code>merge_states</code> combine segments; cascade attention computes a shared prompt prefix once for many requests and merges it with each request’s private suffix
Paged-KV decode kernels	Split boundaries are aligned to KV pages; each CTA walks its slice of the block table
Context / ring parallelism	Each GPU holds a shard of the KV sequence; the same $\oplus$ merges partial states across devices
FlashDecoding++ (Hong et al.)	Replaces the per-split running max with a precomputed unified max to avoid synchronized rescaling, with a fallback path when values exceed it
LeanAttention	Applies Stream-K-style work partitioning to attention to reduce wave quantization

Short query blocks. Speculative decoding verifies several draft tokens at once, so q has 2–8 rows rather than 1. The same kernel shape applies, with a few query rows per CTA and the same split and merge.

8. Verification and benchmark plan

Functional

Check	Catches
<code>num_splits</code> {1, 2, 3, <code>S_max</code> } all equal the reference	Merge bugs, off-by-one split boundaries
<code>seq_len</code> {1, <code>BLOCK_N-1</code> , <code>BLOCK_N</code> , <code>BLOCK_N+1</code> , <code>chunk.k ± 1</code> }	Tail masks and split edges
Ragged batch with empty splits	0/0 in the empty-split path
Large-logit inputs (scale q)	Missing max subtraction in either kernel
BF16/FP8 cache vs FP32 reference with tolerance	Accumulating in the cache dtype
Return LSE and merge two halves externally	LSE scaling bugs (natural log vs <code>log2</code> , missing $1/\sqrt{d}$)

log2 vs ln: many fast kernels compute `exp2(x * log2(e))` internally and keep the running max in base 2. Whatever the kernel uses internally, the LSE it **returns** must state its base. Merging a base-2 LSE with a natural-log LSE silently reweights the splits.

Performance

Metric	Target
Achieved bandwidth = KV bytes / kernel time	Approaches HBM peak for long contexts
Speedup vs <code>num_splits=1</code> across a (batch, context) sweep	Large at batch 1 and long context; about 1× at large batch
Combine kernel share of time	A few percent at most
Occupancy and eligible warps (Nsight Compute)	No spills from oversized <code>BLOCK_N × D</code> tiles

Benchmark with **realistic shapes**: the KV heads count after GQA, the actual cache dtype, and the batch sizes a server sees at its latency SLO. A sweep made only at batch 64 will “prove” split-KV is useless.

9. Prompting an AI assistant for a decode kernel

Generated attention kernels default to prefill-shaped FlashAttention clones. State the decode contract explicitly:

1. **Shapes**: query `[B, H_q, D]` (or `[B, q_len, H_q, D]` for speculative decode), KV `[B, H_kv, T_max, D]` or a paged pool plus `block_table`, and per-sequence `seq_lens`.
2. **Split-KV**: the number of splits as a runtime argument, an FP32 `(o, lse)` workspace, and a separate combine kernel or a documented fused alternative.
3. **Numerics**: an FP32 accumulator, stable max-subtracted merge, empty split `lse = -inf` and `o = 0`, and the LSE base stated.
4. **Tests first**: an interpreter-mode test against `torch.softmax` covering ragged lengths, empty splits, and large logits, plus an ahead-of-time compile for the deployment architecture.
5. **Metric**: achieved GB/s versus HBM peak, not TFLOP/s.

Pitfalls

Pitfall	Result
Naive $\sum e^{\text{LSE}_j} o_j$ merge	<code>inf</code> and NaN with large logits
Empty split writes <code>acc / 1</code> with <code>1 = 0</code>	NaN that poisons the merged output
FP16 workspace or accumulator	Accuracy loss that grows with context and split count
Splitting at large batch	Extra launch and combine with no gain

Pitfall	Result
Splitting a 300-token cache 32 ways	Splits shorter than one tile, so the overhead dominates
Mixing $\log_2$ and natural-log LSEs	Silently wrong weights, no crash
Benchmarking only TFLOP/s	Misses the point: decode is about bytes per second

Checkpoint

- Explain why a FlashAttention grid underfills the GPU at decode with a small batch.
- Define an attention state and prove that $\oplus$ merges disjoint KV sets exactly.
- Write the max-subtracted merge and say why the naive form fails in FP32.
- Describe what an empty split must write, and why.
- Use the wave-efficiency heuristic to say when *not* to split.
- Name three systems that reuse the same merge operator.

Try this

1. Change `split_kv_reference.py` to merge the splits in random order 100 times and confirm that the result never changes beyond rounding.
2. Make `split_kv_kernel` accumulate in FP16 and measure the error growth against `T_max` under `TRITON_INTERPRET=1`.
3. Add a `block_table` argument so that K and V come from a paged pool, and align the split boundaries to page boundaries.
4. Fuse the combine into kernel 1 using an atomic counter: the last CTA to finish for a (sequence, head) performs the merge. Compare launch counts in Nsight Systems on a real GPU.
5. On a GPU, sweep batch `{1, 4, 16, 64}` and context `{1k, 8k, 64k, 128k}` with `num_splits=1` vs the heuristic, and plot achieved GB/s.

Summary

Decode attention is bandwidth-bound, and at small batch the FlashAttention grid has too few CTAs to use that bandwidth. Flash-Decoding splits the KV sequence across CTAs. Each split produces an attention state (o , LSE), and an associative, max-subtracted merge recombines the states exactly. The kernel engineering lives in the details: FP32 state, empty-split handling, ragged lengths, a split count chosen by wave efficiency, and an LSE convention stated as part of the ABI. Verify on CPU with the Triton interpreter, compile ahead of time for the target GPU, and judge the result in GB/s.

Sources (for further reading)

- Dao, Haziza, Massa, Sizov, “Flash-Decoding for long-context inference” (PyTorch blog / Stanford CRFM, Oct 2023).
- Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning” (2023); FlashAttention repository: `flash_attn_with_kvcache` docs and `num_splits_heuristic` in `csrc/flash_attn/flash_api.cpp`.
- Shah et al., “FlashAttention-3” (2024); `hopper/flash_attn_interface.py` (`num_splits`, `pack_gqa`).
- Ye et al., “FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving” (arXiv:2501.01005); FlashInfer docs: “Attention States and Recursive Attention”, `flashinfer.cascade.merge_state`.
- Hong et al., “FlashDecoding++: Faster Large Language Model Inference on GPUs” (arXiv:2311.01282).
- Milakov & Gimelshein, “Online normalizer calculation for softmax” (2018).
- Triton documentation: `TRITON_INTERPRET=1` interpreter mode and `triton.compile / GPUPTarget`.

Part V: AI-Written Kernels

DeepSeek: FlashMLA & DeepGEMM

DeepSeek: FlashMLA & DeepGEMM

DeepSeek stunned the AI industry by open-sourcing compact, ultra-efficient kernels that outcompeted monolithic enterprise libraries.

FlashMLA: Multi-Head Latent Attention

DeepSeek-V3 and R1 utilize Multi-Head Latent Attention (MLA) to compress KV caches into low-dimensional latent spaces. Standard attention kernels cannot handle this asymmetrical projection efficiently. FlashMLA is custom-tuned to decode latent vectors directly into shared memory, cutting KV memory bandwidth consumption by over 70%.

DeepGEMM: Clean Architecture over Brute Force

In fewer than 300 lines of readable, modular CUDA code, **DeepGEMM** implements fine-grained FP8 matrix multiplication with grouped scaling. By avoiding deeply nested template metaprogramming and focusing purely on optimal software pipelining and TMA scheduling, DeepGEMM matches and often beats commercial closed-source GEMM kernels.

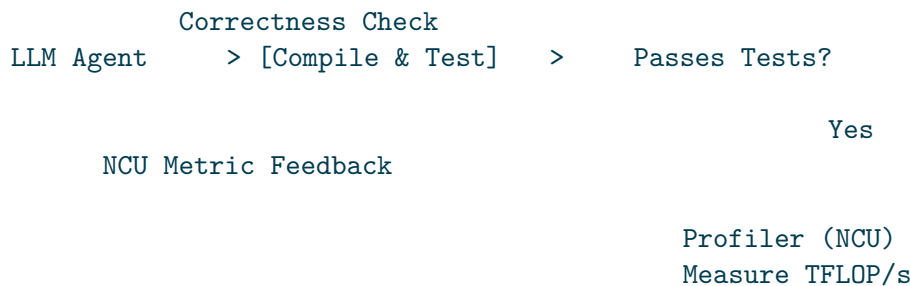
LLM-Generated Kernels: The Agent + Profiler Loop

LLM-Generated Kernels: The Agent + Profiler Loop

We are entering a transformative era where AI models generate, optimize, and verify GPU kernels autonomously.

The Agentic Optimization Loop

[Prompt / Specification]



Google DeepMind AlphaEvolve & KernelBench

- **KernelBench:** An automated evaluation benchmark testing whether language models can generate numerically correct and high-performance CUDA/Triton kernels across realistic neural network primitives.
- **DeepMind AlphaEvolve:** An evolutionary coding agent combining LLM code mutations with real hardware profiler feedback. AlphaEvolve discovered non-trivial memory layout schedules that sped up production attention kernels by ~32%.

The Role of the Human Kernel Engineer

The engineer's job is not becoming obsolete; it is elevating: - Defining invariants, numerical bounds, and correctness testbenches. - Providing architectural guidance and identifying hardware bottlenecks. - Interpreting profiler telemetry to steer the agent's exploration away from local minima.

Part VI: Kernel Engineering Capstone Projects

Crusoe Capstone Specifications & Problem Tracks

Crusoe Capstone Specifications & Problem Tracks

The capstone project requires building a production-ready GPU kernel addressing genuine infrastructure challenges.

Problem Track 1: Fused Sparse Attention

Implement a fused attention kernel supporting block-sparse patterns (e.g. BigBird or sliding window + global tokens). The kernel must: - Skip computation of zero-weight blocks entirely. - Maintain online softmax correctness across sparse blocks. - Outperform standard dense PyTorch attention by at least 2x on sequences >16k.

Problem Track 2: Quantized FP4/FP8 GEMM with Dynamic Dequantization

Build a high-throughput matrix multiply kernel that ingests FP8 or INT4 quantized weights and dequantizes on-the-fly inside SM registers before execution.

Problem Track 3: High-Throughput Speculative Decoding Verification Kernel

Design a batched verification kernel for speculative decoding that verifies K draft tokens concurrently against a target model distribution in a single fused pass.

Verification, Profiling, and Benchmarking Harness

Verification, Profiling, and Benchmarking Harness

Every kernel submission must pass a three-tiered validation suite:

1. Correctness Verification

Compare kernel output tensors against double-precision PyTorch / FP64 reference outputs across edge-case dimensions, non-multiple-of-32 matrix sizes, and extreme numerical scales (10^{-5} to 10^5). Maximum allowable relative error:

$$\frac{\|Y_{\text{kernel}} - Y_{\text{ref}}\|_2}{\|Y_{\text{ref}}\|_2} < 10^{-3}$$

2. CUDA Event Benchmarking

Measure kernel execution time using `cudaEventRecord` over 1,000 warm iterations, discarding outliers:

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);
for (int i = 0; i < 1000; ++i) {
    launch_kernel<<<grid, block>>>(...);
}
cudaEventRecord(stop);
cudaEventSynchronize(stop);

float ms = 0;
cudaEventElapsedTime(&ms, start, stop);
printf("Average execution time: %f ms\n", ms / 1000.0f);
```

Demo Day Presentation & Portfolio

Demo Day Presentation & Portfolio

Presenting your work effectively is as critical as writing fast code.

Deliverables Checklist

1. **GitHub Repository:** Clean, modular repository containing your kernel source, CMake / setup.py build configuration, and automated test runners.
2. **Performance Roofline Graph:** A generated plot showing where your kernel sits relative to theoretical memory and compute ceilings across varying matrix dimensions.
3. **Speedup Comparison Table:** Side-by-side throughput (TFLOP/s) comparisons against naive CUDA, PyTorch `torch.compile`, and official vendor libraries (cuBLAS / cuDNN).
4. **Technical Defense:** Prepare concise explanations for architectural decisions: tile sizes, shared memory allocations, register counts per thread, and how AI-assisted iteration shaped the final solution.